Karakalpak speech recognition

Pipeline walkthrough: every command in order, with the code it runs

wav2vec2 XLS-R 300M + LoRA · KenLM decoding · repository `AI & ML` · generated 2026-10-15 · results from the run on the server spark-f886 (NVIDIA GB10).

The sequence of actions

Step	Where	Command	What it does	Time
1	Mac	`python scripts/build_final_datasets.py --overwrite`	Builds data/final/stt and data/final/hf; logs every text change	≈ 20 s
2	Mac	`python scripts/validate_final_datasets.py`	37 checks against the sources	≈ 40 s
3	Mac → server	`rsync -avc …`	Copies code, data and docs	minutes
4	Server	`python src/vocab.py`	Builds the 35-token vocabulary	seconds
5	Server	`nohup python src/train.py > outputs/raw_logs/train_<date>.log 2>&1 &`	LoRA fine-tuning; validation WER/CER per epoch	2 h 31 min
5b	Server	(automatic at the end of step 5)	Greedy WER/CER on both test sets	≈ 3 min
6	Server	`LD_LIBRARY_PATH=… python src/lm.py`	4-gram KenLM from the train text	seconds
7	Server	`python src/decode.py`	3 decoders compared; tunes α, β	≈ 4 min
8	Server	`python scripts/prepare_benchmarks.py` · `python src/benchmark.py <csv>`	Scores other projects' test sets	≈ 3 min each

How to read this document

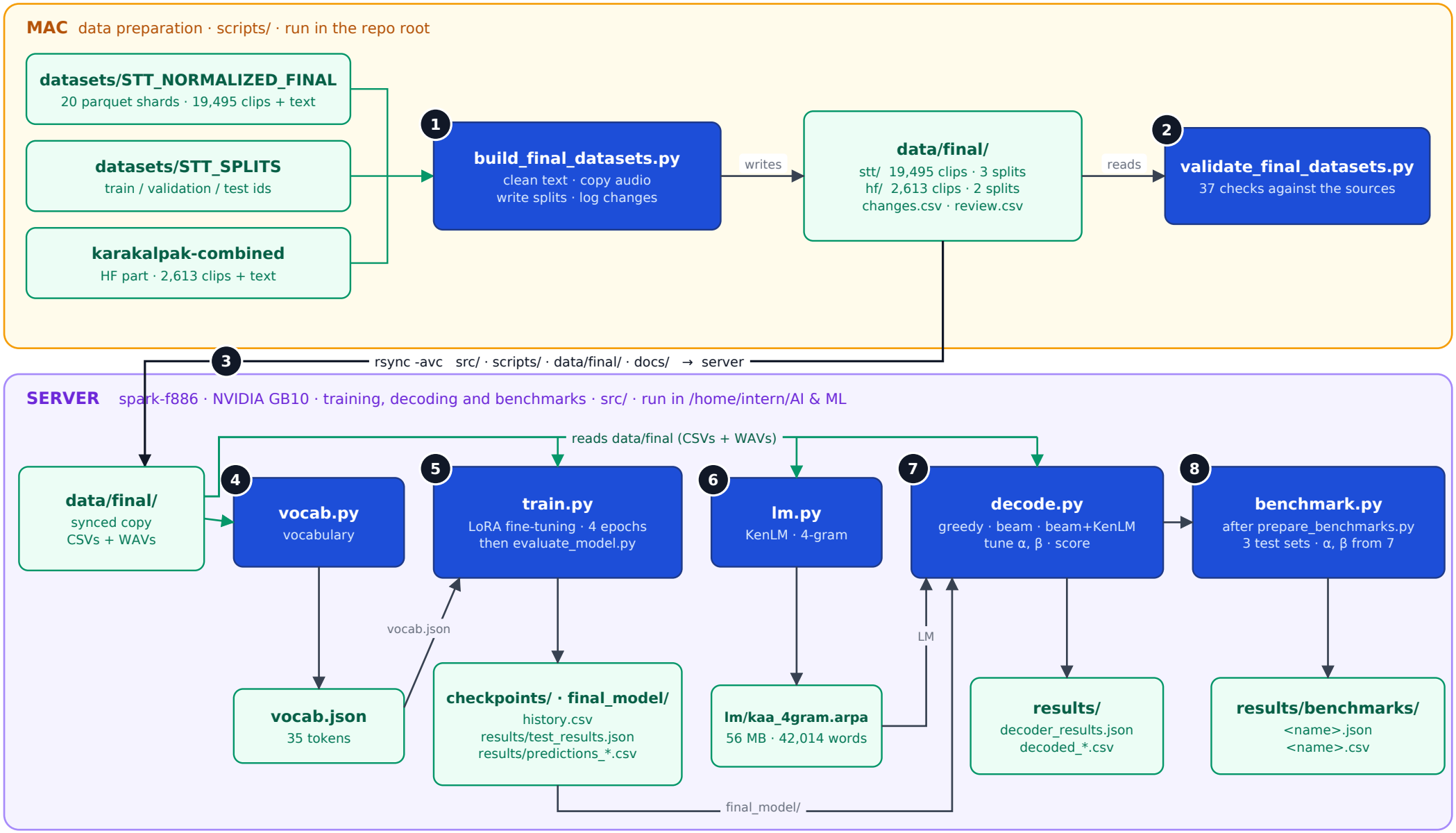
- Each step lists the **command**, **where** it runs, what it **reads** and **writes**, and its **call chain**: which functions run, in which file, in which order.
- A **Functions** table describes every function the step uses. The **Code** that follows is copied from the repository files when this PDF was generated, with the original line numbers; a `·` in the line-number column means a long line continues. The code has no comments: the names say what each part does.
- Code that several steps share is shown once, at its first use; later steps point back to it.
- Pages 2–3 and the start of steps 5 and 7 have diagrams; the same diagrams are in `asr_pipeline_map.pdf`.
- The code was simplified on 2026-10-14 (comments removed, clearer names, small functions). The results come from the run just before; the logic is unchanged, but the simplified code has only been checked statically, not run.

Contents

Step 1 · Build the training data . . . 4
Step 2 · Validate the data . . . 9
Step 3 · Copy everything to the server . . . 14
Step 4 · Build the character vocabulary . . . 14
Step 5 · Train the model . . . 16
Step 5b · Score the test sets . . . 22
Step 6 · Build the KenLM language model . . . 23
Step 7 · Decode and compare: greedy, beam search, KenLM . . . 24
Step 8 · Test on data from other projects . . . 29
Appendix A · config.py: every setting . . . 33
Appendix B · Quick local run on the Mac . . . 35
Appendix C · Results summary . . . 35

End-to-end pipeline

What runs where, in order. The numbers match the steps in the walkthrough PDF.

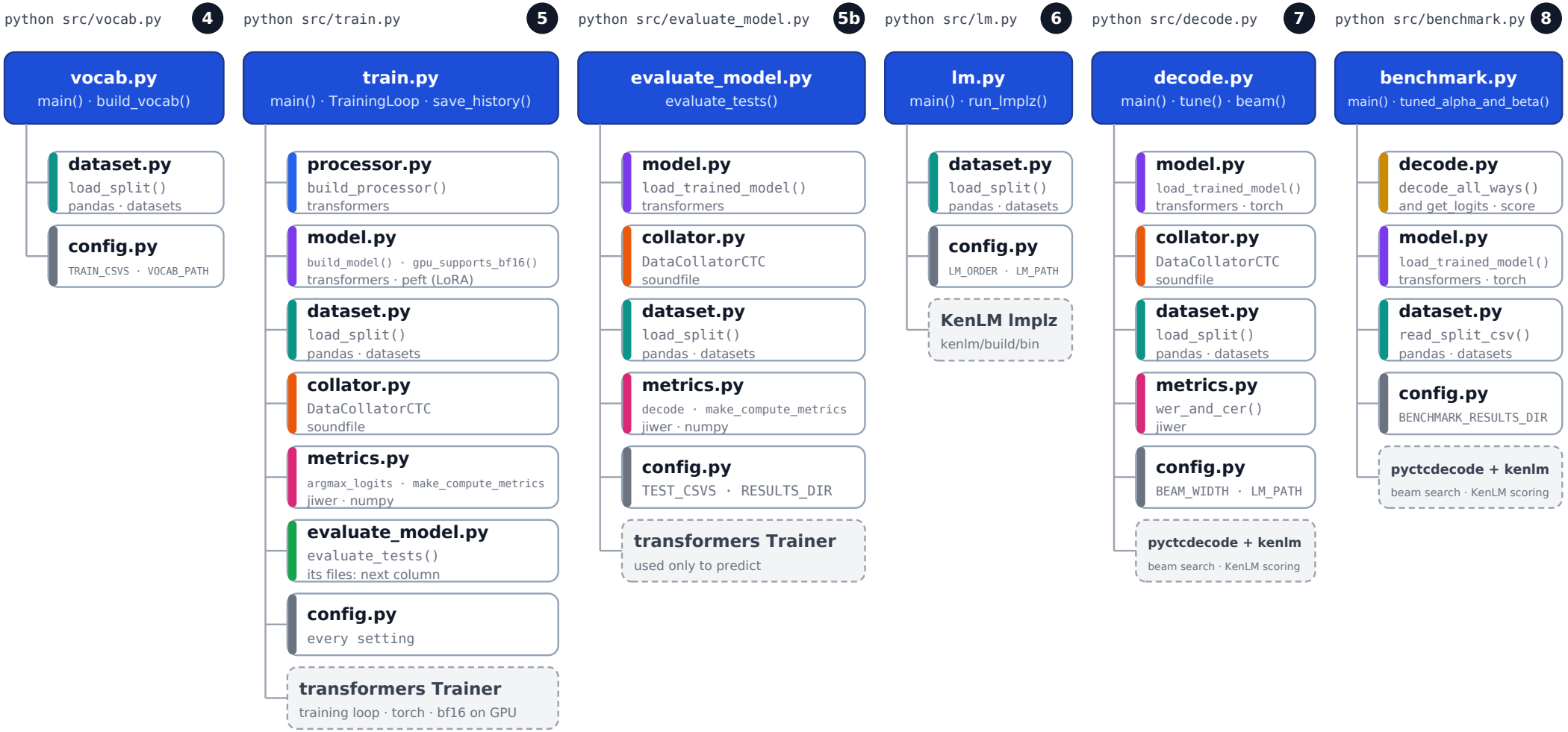


lm.py needs only the train text, so step 6 can run any time; decode.py needs the trained model and the LM; benchmark.py also needs decode.py's α and β.

script you run · data / files · 1 step number · reads / writes

Which files each command uses

Each column is one command. Under it: the src/ files it imports (coloured by file) and the outside tools it needs.



dataset.py, collator.py, metrics.py and model.py are shared by several commands; every file reads config.py.
Step 8 also needs scripts/prepare_benchmarks.py, run once before it to build data/benchmarks/.

Files: config.py · dataset.py · collator.py · metrics.py · processor.py · model.py · evaluate_model.py · decode.py

Step 1 · Build the training data

Mac · repository root

Command	`.venv/bin/python scripts/build_final_datasets.py --overwrite`
Time	about 20 seconds
Reads	`data/datasets/STT_NORMALIZED_FINAL/*.parquet` (20 shards: id, audio, text_normalized) · `data/datasets/STT_SPLITS/{train,validation,test}.csv` · `data/karakalpak-combined/metadata.csv`, `provenance/metadata.csv` and its HF WAV files
Writes	`data/final/{stt,hf}/<split>/audio/*.wav` · `<split>.csv` · `metadata.csv` · `changes.csv` · `review.csv` · `provenance/quality.csv` · `split_summary.json` · `README.md` · `data/final/summary.json`

What it does

Turns the raw corpora into two train-ready datasets: `stt` (news, 93 h) and `hf` (literary, 3.5 h). Every WAV is copied byte for byte. Every transcript is cleaned so it uses only the 2016 Karakalpak Latin alphabet, and leftovers from an earlier digit-to-word conversion are repaired; every edit is recorded with the original text. Everything is built in `data/final.partial`, which replaces `data/final` only when all of it succeeded.

Call chain

```
python scripts/build_final_datasets.py --overwrite
└─ main()
   ├─ fresh_folder(data/final.partial)
   ├─ build_stt(stage, records)          STT: 20 parquet shards + STT_SPLITS
   │  ├─ stt_split_assignments()         split of every id
   │  └─ add_record()                    × 19,495
   │     ├─ pcm_frames()                 must be 16 kHz mono 16-bit
   │     ├─ clean_text()                 í → ı, then:
   │     │  └─ fix_number_words()        split / spelling / suffix repairs
   │     │     ├─ is_split_off_suffix()
   │     │     └─ attach_suffix() → dative() → last_vowel_is_front()
   │     └─ quality() → decibels()       dBFS, clipping, characters per second
   ├─ build_hf(stage, records)           HF rows of karakalpak-combined
   │  └─ add_record()                    × 2,613, same sub-steps as above
   ├─ check_splits() · check_leakage()   expected splits; nothing shared between splits
   ├─ write_dataset()                    × 2 (stt, hf)
   │  ├─ write_csv()                     split CSVs, metadata, changes, quality
   │  ├─ review_rows()                   rows a person should listen to
   │  ├─ summarize() → hours()
   │  └─ write_json()                    split_summary.json
   └─ write_json(summary.json) · README.md · fresh_folder(data/final) · stage.rename()
```

Result of this run

Dataset	Clips	Hours	Splits	Transcripts changed
stt	19,495	93.13	train 15,595 · validation 1,950 · test 1,950	2,115 (suffix 799 · spelling 1,473 · split 28 · í→ı 17) · review.csv 284 rows
hf	2,613	3.48	train 2,090 · test 523	1 (í→ı)

Functions

All in `scripts/build_final_datasets.py`.

Function	Lines	What it does
`main()`	318–342	Parses `--overwrite`, builds into a fresh `data/final.partial`, checks splits and leakage, writes both datasets, then swaps the staging folder into place.
`fresh_folder(path)`	312–315	Deletes the folder if it exists.
`build_stt(stage, records)`	227–240	Streams each STT parquet shard, checks its text matches STT_SPLITS, and calls `add_record` for every clip as `stt_<id>.wav`.
`stt_split_assignments()`	217–224	id → (split, text) from STT_SPLITS; an id in two splits is an error.
`build_hf(stage, records)`	243–251	Keeps only the HF rows of karakalpak-combined (Mendeley is skipped) and calls `add_record` with their WAV, transcript and original split.

Function	Lines	What it does
`add_record(...)`	191–214	Per clip: validates the audio, cleans the text, refuses characters outside `ALPHABET`, writes the WAV unchanged, stores one metadata row.
`pcm_frames(data, label)`	162–169	Requires 16 kHz mono 16-bit, complete audio; returns the PCM bytes.
`clean_text(text)`	152–159	`LETTER_REPLACEMENTS` (í → ı), then `fix_number_words`; returns the text and the rules applied.
`fix_number_words(text)`	133–149	Word by word: splits `ABBREVIATION_GLUED_TO_NUMBER` (`covidon` → `covid on`), applies `NUMBER_SPELLINGS` (`jıyırma` → `jigirma`), re-attaches split-off suffixes.
`is_split_off_suffix(word, previous_word)`	121–122	`ǵı`/`gi` after any word, or a number suffix right after a number word.
`attach_suffix(word, suffix)`	125–130	Joins the suffix; a dative after a number gets the harmonised form from `dative()` (`júz ǵa` → `júzge`).
`dative(word)`	114–118	Front last vowel → ge/ke, back → ǵa/qa; voiceless last sound → qa/ke.
`last_vowel_is_front(word)`	109–111	Vowel harmony test used by `dative()`.
`quality(...) · decibels(value)`	176–188	Duration, peak and RMS level (dBFS), clipped samples, characters per second for `provenance/quality.csv`.
`check_splits(records)`	254–258	Every row is in a split its dataset has.
`check_leakage(records)`	261–269	No audio (PCM hash) and no sentence in two splits, across both datasets.
`write_dataset(folder, source, rows)`	298–309	Writes one dataset's split CSVs, metadata, changes, quality, review list, summary and README.
`review_rows(rows)`	288–295	Rows a script can't fix safely: a number said twice (`REPEATED_NUMBER`) or a lone suffix after a name.
`summarize(rows, splits) · hours(rows)`	276–285	Files, hours, changed transcripts, rules and characters per dataset.
`read_csv · write_csv · write_json`	—	Small file helpers.

Code

Settings: paths, alphabet, repair rules, columns · scripts/build_final_datasets.py · lines 17–50

```
17  ROOT = Path(__file__).resolve().parents[1]
18  DATA = ROOT / "data"
19  STT = DATA / "datasets" / "STT_NORMALIZED_FINAL"
20  STT_SPLITS = DATA / "datasets" / "STT_SPLITS"
21  COMBINED = DATA / "karakalpak-combined"
22  FINAL = DATA / "final"
23
24  DATASETS = {"stt": ("train", "validation", "test"), "hf": ("train", "test")}
25  ALPHABET = set("aábcdefgǵhiıjklmnńoópqrstuúvwxyz ")
26  LETTER_REPLACEMENTS = (("í", "ı"),)
27
28  NUMBER_SPELLINGS = {"jıyırma": "jigirma", "jıyırmanshı": "jigirmanshı", "elli": "eliw", "mın": "mıń"}
29  NUMBER_WORDS = set("bir eki úsh tórt bes altı jeti segiz toǵız on jigirma otız qırıq qırq eliw alpıs jetpis "
30                     "seksen toqsan júz mıń million milliard".split())
31  SUFFIXES_AFTER_NUMBERS = set("ǵa ge qa ke ga ka sı si ın in nshı nshi ınshı inshi nıń nińdıń diń tıń tiń "
32                               "dı di tı ti dan den tan ten nan nen".split())
33  SUFFIXES_AFTER_ANY_WORD = {"ǵı", "gi"}
34  DATIVE_SUFFIXES = {"ǵa", "ge", "qa", "ke", "ga", "ka"}
35  FRONT_VOWELS = "áeıóú"
36  ALL_VOWELS = "aáeıioóuú"
37  VOICELESS_CONSONANTS = "ptkqsfxhc"
38
39  NUMBER_WORD_PATTERN = "|".join(sorted(NUMBER_WORDS, key=len, reverse=True))
40  ABBREVIATION_GLUED_TO_NUMBER = re.compile(rf"(covid|top|cop)({NUMBER_WORD_PATTERN})")
41  REPEATED_NUMBER = re.compile(rf"(?<!\S)((?:{NUMBER_WORD_PATTERN})(?: (?:{NUMBER_WORD_PATTERN}))*) \1(?!\S)")
42
43  TRAIN_FIELDS = ["audio_path", "text", "duration"]
44  METADATA_FIELDS = [*TRAIN_FIELDS, "split", "source_id", "source_location", "text_original",
45                     "text_rules", "pcm_sha256"]
46  CHANGE_FIELDS = ["audio_path", "split", "source_id", "source_location", "text_rules",
47                   "text_original", "text"]
48  REVIEW_FIELDS = ["audio_path", "split", "issue", "text"]
49  QUALITY_FIELDS = ["audio_path", "duration", "peak_dbfs", "rms_dbfs", "clipped_fraction",
50                    "chars_per_second"]
```

main() · scripts/build_final_datasets.py · lines 318–342

```python
def main():
    parser = argparse.ArgumentParser(description="Build data/final/stt and data/final/hf from the raw corpora."
        )
    parser.add_argument("--overwrite", action="store_true", help="replace an existing data/final")
    args = parser.parse_args()
    if FINAL.exists() and not args.overwrite:
        raise SystemExit(f"{FINAL} exists; pass --overwrite to rebuild it")

    stage = fresh_folder(FINAL.with_name(FINAL.name + ".partial"))
    records = {source: [] for source in DATASETS}
    build_stt(stage, records)
    build_hf(stage, records)
    check_splits(records)
    check_leakage(records)

    result = {
        "rules": [f"{old}->{new}" for old, new in LETTER_REPLACEMENTS],
        "alphabet": "".join(sorted(ALPHABET - {" "})),
        "datasets": {source: write_dataset(stage / source, source, rows) for source, rows in records.items()},
    }
    write_json(stage / "summary.json", result)
    (stage / "README.md").write_text(FINAL_README, encoding="utf-8")

    fresh_folder(FINAL)
    stage.rename(FINAL)
    print(json.dumps(result, ensure_ascii=False, indent=2))
```

fresh_folder() · scripts/build_final_datasets.py · lines 312–315

```python
def fresh_folder(path):
    if path.exists():
        shutil.rmtree(path)
    return path
```

build_stt() · scripts/build_final_datasets.py · lines 227–240

```python
def build_stt(stage, records):
    assignments = stt_split_assignments()
    seen = set()
    for shard in sorted(STT.glob("*.parquet")):
        for record in pq.read_table(shard, columns=["id", "audio", "text_normalized"]).to_pylist():
            key = str(record["id"])
            split, split_text = assignments[key]
            if split_text != record["text_normalized"]:
                raise ValueError(f"STT id {key}: split CSV text differs from parquet")
            add_record(stage, records, "stt", split, f"stt_{record['id']:06d}.wav", record["audio"],
                       record["text_normalized"], key, f"datasets/STT_NORMALIZED_FINAL/{shard.name}#id={key}")
            seen.add(key)
    if seen != set(assignments):
        raise ValueError("STT parquet ids and STT_SPLITS ids differ")
```

stt_split_assignments() · scripts/build_final_datasets.py · lines 217–224

```python
def stt_split_assignments():
    assignments = {}
    for split in DATASETS["stt"]:
        for row in read_csv(STT_SPLITS / f"{split}.csv"):
            if row["id"] in assignments:
                raise ValueError(f"STT id {row['id']} appears in more than one split")
            assignments[row["id"]] = (split, row["text_normalized"])
    return assignments
```

build_hf() · scripts/build_final_datasets.py · lines 243–251

```python
def build_hf(stage, records):
    detailed = {row["audio_path"]: row for row in read_csv(COMBINED / "provenance/metadata.csv")}
    for row in read_csv(COMBINED / "metadata.csv"):
        source_row = detailed[row["audio_path"]]
        if source_row["source"] != "hf":
            continue
        path = COMBINED / row["audio_path"]
        add_record(stage, records, "hf", row["split"], path.name, path.read_bytes(), row["text"],
                   source_row["sample_id"], f"karakalpak-combined/{row['audio_path']}")
```

add_record() · scripts/build_final_datasets.py · lines 191–214

```python
def add_record(stage, records, source, split, name, data, text_original, source_id, location):
    frames = pcm_frames(data, location)
    text, rules = clean_text(text_original)
    outside = set(text) - ALPHABET
    if outside:
        raise ValueError(f"{location}: characters outside the alphabet {sorted(outside)}")

    audio_path = f"{split}/audio/{name}"
    target = stage / source / audio_path
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_bytes(data)

    records[source].append({
        "audio_path": audio_path,
        "text": text,
        "duration": len(frames) / 32000,
        "split": split,
        "source_id": source_id,
        "source_location": location,
        "text_original": text_original,
        "text_rules": rules,
        "pcm_sha256": hashlib.sha256(frames).hexdigest(),
        "quality": quality(audio_path, frames, text),
    })
```

pcm_frames() · scripts/build_final_datasets.py · lines 162–169

```python
def pcm_frames(data, label):
    with wave.open(io.BytesIO(data), "rb") as audio:
        if (audio.getframerate(), audio.getnchannels(), audio.getsampwidth()) != (16000, 1, 2):
            raise ValueError(f"Expected mono 16 kHz PCM16: {label}")
        frames = audio.readframes(audio.getnframes())
    if len(frames) != audio.getnframes() * 2 or not frames:
        raise ValueError(f"Truncated or empty audio: {label}")
    return frames
```

clean_text() · scripts/build_final_datasets.py · lines 152–159

```python
def clean_text(text):
    applied = []
    for old, new in LETTER_REPLACEMENTS:
        if old in text:
            text = text.replace(old, new)
            applied.append(f"{old}->{new}")
    text, repairs = fix_number_words(text)
    return text, ";".join(applied + repairs)
```

fix_number_words() · scripts/build_final_datasets.py · lines 133–149

```python
def fix_number_words(text):
    words, repairs = [], set()
    for word in text.split():
        glued = ABBREVIATION_GLUED_TO_NUMBER.fullmatch(word)
        if glued:
            words.append(glued.group(1))
            word = glued.group(2)
            repairs.add("split")
        if word in NUMBER_SPELLINGS:
            word = NUMBER_SPELLINGS[word]
            repairs.add("spelling")
        if words and is_split_off_suffix(word, words[-1]):
            words[-1] = attach_suffix(words[-1], word)
            repairs.add("suffix")
        else:
            words.append(word)
    return " ".join(words), sorted(repairs)
```

is_split_off_suffix() · scripts/build_final_datasets.py · lines 121–122

```python
def is_split_off_suffix(word, previous_word):
    return word in SUFFIXES_AFTER_ANY_WORD or (word in SUFFIXES_AFTER_NUMBERS and previous_word in NUMBER_WORDS
        )
```

attach_suffix() · scripts/build_final_datasets.py · lines 125–130

```python
def attach_suffix(word, suffix):
    if word.endswith(suffix):
        return word
    if suffix in DATIVE_SUFFIXES and word in NUMBER_WORDS:
        return word + dative(word)
    return word + suffix
```

dative() · scripts/build_final_datasets.py · lines 114–118

```python
def dative(word):
    ends_voiceless = word[-1] in VOICELESS_CONSONANTS
    if last_vowel_is_front(word):
        return "ke" if ends_voiceless else "ge"
    return "qa" if ends_voiceless else "ǵa"
```

last_vowel_is_front() · scripts/build_final_datasets.py · lines 109–111

```python
def last_vowel_is_front(word):
    vowels = [letter for letter in word if letter in ALL_VOWELS]
    return bool(vowels) and vowels[-1] in FRONT_VOWELS
```

quality() · scripts/build_final_datasets.py · lines 176–188

```python
def quality(audio_path, frames, text):
    samples = np.frombuffer(frames, dtype="<i2").astype(np.float64)
    duration = len(samples) / 16000
    peak = np.abs(samples).max()
    rms = np.sqrt(np.mean(samples ** 2))
    return {
        "audio_path": audio_path,
        "duration": f"{duration:.4f}",
        "peak_dbfs": f"{decibels(max(peak, 1)):.2f}",
        "rms_dbfs": f"{decibels(max(rms, 1e-9)):.2f}",
        "clipped_fraction": f"{np.mean(np.abs(samples) >= 32767):.6f}",
        "chars_per_second": f"{len(text.replace(' ', '')) / duration:.2f}",
    }
```

decibels() · scripts/build_final_datasets.py · lines 172–173

```python
def decibels(value):
    return 20 * np.log10(value / 32768)
```

check_splits() · scripts/build_final_datasets.py · lines 254–258

```python
def check_splits(records):
    for source, rows in records.items():
        unexpected = {row["split"] for row in rows} - set(DATASETS[source])
        if unexpected:
            raise ValueError(f"{source} has unexpected splits {unexpected}")
```

check_leakage() · scripts/build_final_datasets.py · lines 261–269

```python
def check_leakage(records):
    rows = [row for part in records.values() for row in part]
    for key in ("pcm_sha256", "text"):
        splits = defaultdict(set)
        for row in rows:
            splits[row[key]].add(row["split"])
        crossing = sum(len(value) > 1 for value in splits.values())
        if crossing:
            raise ValueError(f"{crossing} {key} groups cross split boundaries")
```

write_dataset() · scripts/build_final_datasets.py · lines 298–309

```python
def write_dataset(folder, source, rows):
    rows.sort(key=lambda row: row["audio_path"])
    for split in DATASETS[source]:
        write_csv(folder / f"{split}.csv", [row for row in rows if row["split"] == split], TRAIN_FIELDS)
    write_csv(folder / "metadata.csv", rows, METADATA_FIELDS)
    write_csv(folder / "changes.csv", [row for row in rows if row["text"] != row["text_original"]], CHANGE_FIEL
    DS)
    write_csv(folder / "provenance/quality.csv", [row["quality"] for row in rows], QUALITY_FIELDS)
    write_csv(folder / "review.csv", review_rows(rows), REVIEW_FIELDS)
    result = summarize(rows, DATASETS[source])
    write_json(folder / "split_summary.json", result)
    (folder / "README.md").write_text(DATASET_README[source], encoding="utf-8")
    return result
```

review_rows() · scripts/build_final_datasets.py · lines 288–295

```python
def review_rows(rows):
    review = []
    for row in rows:
        if REPEATED_NUMBER.search(row["text"]):
            review.append({**row, "issue": "repeated number"})
        if any(word in SUFFIXES_AFTER_NUMBERS for word in row["text"].split()[1:]):
            review.append({**row, "issue": "standalone suffix"})
    return review
```

summarize() · scripts/build_final_datasets.py · lines 276–285

```python
def summarize(rows, splits):
    by_split = {split: [row for row in rows if row["split"] == split] for split in splits}
    return {
        "splits": {split: {"files": len(part), "hours": hours(part)} for split, part in by_split.items()},
        "files": len(rows),
        "hours": hours(rows),
        "transcripts_changed": sum(row["text"] != row["text_original"] for row in rows),
        "rules": dict(Counter(rule for row in rows for rule in row["text_rules"].split(";") if rule)),
        "characters": "".join(sorted({char for row in rows for char in row["text"]} - {" "})),
    }
```

hours() · scripts/build_final_datasets.py · lines 272–273

```python
def hours(rows):
    return round(sum(row["duration"] for row in rows) / 3600, 3)
```

read_csv() · scripts/build_final_datasets.py · lines 92–94

```python
def read_csv(path):
    with path.open(encoding="utf-8-sig", newline="") as handle:
        return list(csv.DictReader(handle))
```

write_csv() · scripts/build_final_datasets.py · lines 97–102

```python
def write_csv(path, rows, fields):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields, extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)
```

write_json() · scripts/build_final_datasets.py · lines 105–106

```python
def write_json(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
```

Step 2 · Validate the data

Mac · repository root

Command	`.venv/bin/python scripts/validate_final_datasets.py`
Time	about 40 seconds
Reads	`data/final` and every original source again · imports the rules and `clean_text` from `build_final_datasets.py` · imports `src/config.py` and `src/dataset.py`
Writes	nothing; prints PASS or FAIL per check and exits with an error if any failed

What it does

Checks the built data independently of the build: it re-reads the original sources byte by byte, so a bug in the build would show up here, and it loads samples through `src/dataset.py` like training does.

Call chain

```
python scripts/validate_final_datasets.py
└─ main()                                          every check calls check(): prints PASS / FAIL
   ├─ check_folders()                              only stt/ and hf/ exist
   ├─ for stt and hf:
   │  ├─ check_layout()                            CSV rows ↔ WAV files ↔ metadata.csv
   │  ├─ check_audio() → wav(), sha256()           format, duration, PCM SHA256
   │  ├─ check_text() → clean_text()               alphabet, no digits, only documented edits
   │  │  └─ has_leftover_number_words()            no repair left undone
   │  └─ check_change_log()                        changes.csv lists exactly the edits
   ├─ check_stt_sources() → stt_split_assignments()  identical to the parquet; TOZALANGAN = NORMALIZED
   ├─ check_hf_sources()                           identical to karakalpak-combined
   ├─ check_leakage()                              nothing shared between splits
   └─ check_loader() → src/dataset.load_split()    reads 100 samples like training
```

Output of the last run

Recorded before the simplification; the check messages are unchanged.

```
PASS data/final holds exactly ['hf', 'stt']
PASS stt: metadata.csv has the expected columns
PASS stt: split folders are ('train', 'validation', 'test')
PASS stt/train: 15595 CSV rows match the WAV files on disk exactly
PASS stt/validation: 1950 CSV rows match the WAV files on disk exactly
PASS stt/test: 1950 CSV rows match the WAV files on disk exactly
PASS stt: split CSVs together equal metadata.csv
PASS stt: all 19495 WAVs are complete mono 16 kHz PCM16 with matching duration and PCM SHA256
PASS stt: no empty transcripts or irregular spaces
PASS stt: every character is in the 2016 Karakalpak alphabet {}
PASS stt: no digits
PASS stt: transcripts equal the originals with only the documented fixes
PASS stt: no split suffixes after numbers, no 'haqqında ǵı', no glued 'covidon', no jıyırma/elli/mın
PASS stt: review.csv present
PASS stt: changes.csv lists exactly the 2115 edited transcripts
PASS hf: metadata.csv has the expected columns
PASS hf: split folders are ('train', 'test')
PASS hf/train: 2090 CSV rows match the WAV files on disk exactly
PASS hf/test: 523 CSV rows match the WAV files on disk exactly
PASS hf: split CSVs together equal metadata.csv
PASS hf: all 2613 WAVs are complete mono 16 kHz PCM16 with matching duration and PCM SHA256
PASS hf: no empty transcripts or irregular spaces
PASS hf: every character is in the 2016 Karakalpak alphabet {}
PASS hf: no digits
PASS hf: transcripts equal the originals with only the documented fixes
PASS hf: no split suffixes after numbers, no 'haqqında ǵı', no glued 'covidon', no jıyırma/elli/mın
PASS hf: review.csv present
PASS hf: changes.csv lists exactly the 1 edited transcripts
PASS stt: all 19495 parquet ids present once
PASS stt: every WAV is byte-identical to its parquet audio
PASS stt: original text and split match STT_SPLITS and parquet
PASS TOZALANGAN_TRAIN_FINAL equals STT_NORMALIZED_FINAL audio and cleaned_text (19495/19495)
PASS hf: all 2613 HF recordings present, nothing else
PASS hf: audio bytes, original text and train/test split unchanged
PASS no pcm_sha256 value crosses splits, within or across datasets
PASS no text value crosses splits, within or across datasets
PASS src/dataset.py load_split reads 100 random samples from every training CSV

0 failed checks
```

Functions

All in `scripts/validate_final_datasets.py`.

Function	Lines	What it does
`main()`	199–212	Runs every check in order, prints the number of failures, exits 1 if any failed.
`check(condition, message)`	21–24	Prints PASS or FAIL and remembers failures.
`check_folders()`	45–47	`data/final` holds exactly `stt/` and `hf/`.
`check_layout(source)`	50–69	Columns are right, each split CSV lists exactly the WAVs on disk, and the split CSVs together equal `metadata.csv`.
`check_audio(source, metadata)`	72–80	Every WAV is complete 16 kHz mono 16-bit with the duration and PCM hash in `metadata.csv`.
`check_text(source, metadata)`	90–109	Only alphabet letters, no digits, and each transcript equals `clean_text(original)`.
`has_leftover_number_words(words)`	83–87	True if any number-word repair was left undone.
`check_change_log(source, metadata)`	112–118	`changes.csv` lists exactly the edited rows.
`check_stt_sources(metadata)`	130–157	Every STT WAV is byte-identical to the parquet, and TOZALANGAN equals NORMALIZED.
`stt_split_assignments()`	121–127	Split and text of every STT id, read again independently of the build.
`check_hf_sources(metadata)`	160–170	Every HF clip identical to karakalpak-combined.
`check_leakage(final)`	173–179	No PCM hash and no sentence in two splits.

Function	Lines	What it does
`check_loader()`	182–196	Loads 20 random clips from every training CSV with `src/dataset.py`.
`read_csv · wav · sha256`	—	Helpers: read a CSV with its columns, read WAV bytes, hash bytes.

Code

main() · scripts/validate_final_datasets.py · lines 199–212

```
199 def main():
200     check_folders()
201     final = {}
202     for source in DATASETS:
203         final[source] = check_layout(source)
204         check_audio(source, final[source])
205         check_text(source, final[source])
206         check_change_log(source, final[source])
207     check_stt_sources(final["stt"])
208     check_hf_sources(final["hf"])
209     check_leakage(final)
210     check_loader()
211     print(f"\n{len(failures)} failed checks")
212     sys.exit(1 if failures else 0)
```

check() · scripts/validate_final_datasets.py · lines 21–24

```
21 def check(condition, message):
22     print(("PASS " if condition else "FAIL ") + message)
23     if not condition:
24         failures.append(message)
```

read_csv() · scripts/validate_final_datasets.py · lines 27–30

```
27 def read_csv(path):
28     with path.open(encoding="utf-8", newline="") as handle:
29         reader = csv.DictReader(handle)
30         return reader.fieldnames, list(reader)
```

wav() · scripts/validate_final_datasets.py · lines 33–38

```
33 def wav(data):
34     with wave.open(io.BytesIO(data), "rb") as audio:
35         fmt = (audio.getframerate(), audio.getnchannels(), audio.getsampwidth())
36         frames = audio.readframes(audio.getnframes())
37         complete = len(frames) == audio.getnframes() * 2
38     return fmt, frames, complete
```

sha256() · scripts/validate_final_datasets.py · lines 41–42

```
41 def sha256(data):
42     return hashlib.sha256(data).hexdigest()
```

check_folders() · scripts/validate_final_datasets.py · lines 45–47

```
45 def check_folders():
46     present = {p.name for p in FINAL.iterdir() if p.is_dir()}
47     check(present == set(DATASETS), f"data/final holds exactly {sorted(DATASETS)}")
```

check_layout() · scripts/validate_final_datasets.py · lines 50–69

```
50 def check_layout(source):
51     folder = FINAL / source
52     fields, metadata = read_csv(folder / "metadata.csv")
53     check(fields == METADATA_FIELDS, f"{source}: metadata.csv has the expected columns")
54     split_folders = {p.name for p in folder.iterdir() if (p / "audio").is_dir()}
55     check(split_folders == set(DATASETS[source]), f"{source}: split folders are {DATASETS[source]}")
56
57     union = []
58     for split in DATASETS[source]:
59         fields, rows = read_csv(folder / f"{split}.csv")
60         on_disk = {f"{split}/audio/{p.name}" for p in (folder / split / "audio").iterdir()}
61         listed = {row["audio_path"] for row in rows}
62         check(fields == TRAIN_FIELDS and on_disk == listed and len(listed) == len(rows),
63               f"{source}/{split}: {len(rows)} CSV rows match the WAV files on disk exactly")
64         union += rows
65
66     simplified = [{k: row[k] for k in TRAIN_FIELDS} for row in metadata]
67     check(sorted(tuple(r.values()) for r in union) == sorted(tuple(r.values()) for r in simplified),
68           f"{source}: split CSVs together equal metadata.csv")
69     return metadata
```

check_audio() · scripts/validate_final_datasets.py · lines 72–80

```
def check_audio(source, metadata):
    bad_format = bad_duration = bad_hash = 0
    for row in metadata:
        fmt, frames, complete = wav((FINAL / source / row["audio_path"]).read_bytes())
        bad_format += fmt != (16000, 1, 2) or not complete or not frames
        bad_duration += abs(len(frames) / 32000 - float(row["duration"])) > 1e-6
        bad_hash += sha256(frames) != row["pcm_sha256"]
    check(bad_format == 0 and bad_duration == 0 and bad_hash == 0,
          f"{source}: all {len(metadata)} WAVs are complete mono 16 kHz PCM16 with matching duration and PCM
              SHA256")
```

has_leftover_number_words() · scripts/validate_final_datasets.py · lines 83–87

```
def has_leftover_number_words(words):
    split_suffix = any(a in NUMBER_WORDS and b in SUFFIXES_AFTER_NUMBERS for a, b in zip(words, words[1:]))
    leftover_word = any(word in SUFFIXES_AFTER_ANY_WORD or word in NUMBER_SPELLINGS
                        or ABBREVIATION_GLUED_TO_NUMBER.fullmatch(word) for word in words)
    return split_suffix or leftover_word
```

check_text() · scripts/validate_final_datasets.py · lines 90–109

```
def check_text(source, metadata):
    empty = spacing = undocumented_edits = leftovers = 0
    outside = defaultdict(int)
    for row in metadata:
        text = row["text"]
        words = text.split()
        empty += not text
        spacing += text != " ".join(words)
        undocumented_edits += (text, row["text_rules"]) != clean_text(row["text_original"])
        leftovers += has_leftover_number_words(words)
        for char in set(text) - ALPHABET:
            outside[char] += 1

    check(empty == 0 and spacing == 0, f"{source}: no empty transcripts or irregular spaces")
    check(not outside, f"{source}: every character is in the 2016 Karakalpak alphabet {dict(outside)}")
    check(not any(c.isdigit() for row in metadata for c in row["text"]), f"{source}: no digits")
    check(undocumented_edits == 0, f"{source}: transcripts equal the originals with only the documented fixes")
    check(leftovers == 0, f"{source}: no split suffixes after numbers, no 'haqqında ǵı', no glued 'covidon', no
         jıyırma/elli/mın")
    fields, _ = read_csv(FINAL / source / "review.csv")
    check(fields == REVIEW_FIELDS, f"{source}: review.csv present")
```

check_change_log() · scripts/validate_final_datasets.py · lines 112–118

```
def check_change_log(source, metadata):
    fields, changes = read_csv(FINAL / source / "changes.csv")
    expected = {row["audio_path"]: row for row in metadata if row["text"] != row["text_original"]}
    matches = all(expected[c["audio_path"]]["text"] == c["text"] and
                  expected[c["audio_path"]]["text_original"] == c["text_original"] for c in changes)
    check(fields == CHANGE_FIELDS and len(changes) == len(expected) and matches,
          f"{source}: changes.csv lists exactly the {len(expected)} edited transcripts")
```

stt_split_assignments() · scripts/validate_final_datasets.py · lines 121–127

```
def stt_split_assignments():
    split_of, split_text = {}, {}
    for split in DATASETS["stt"]:
        for row in read_csv(STT_SPLITS / f"{split}.csv")[1]:
            split_of[row["id"]] = split
            split_text[row["id"]] = row["text_normalized"]
    return split_of, split_text
```

check_stt_sources() · scripts/validate_final_datasets.py · lines 130–157

```python
def check_stt_sources(metadata):
    rows = {row["source_id"]: row for row in metadata}
    split_of, split_text = stt_split_assignments()
    byte_mismatch = text_mismatch = 0
    seen = set()
    normalized = {}
    for shard in sorted(STT.glob("*.parquet")):
        columns = ["id", "audio", "text_normalized", "cleaned_text"]
        for record in pq.read_table(shard, columns=columns).to_pylist():
            key = str(record["id"])
            seen.add(key)
            row = rows[key]
            digest = sha256(record["audio"])
            normalized[key] = (digest, record["cleaned_text"])
            byte_mismatch += sha256((FINAL / "stt" / row["audio_path"]).read_bytes()) != digest
            text_mismatch += row["text_original"] != record["text_normalized"]
            text_mismatch += row["split"] != split_of[key] or split_text[key] != record["text_normalized"]
    check(seen == set(rows) == set(split_of), f"stt: all {len(seen)} parquet ids present once")
    check(byte_mismatch == 0, "stt: every WAV is byte-identical to its parquet audio")
    check(text_mismatch == 0, "stt: original text and split match STT_SPLITS and parquet")

    same = total = 0
    for shard in sorted(CLEANED.glob("*.parquet")):
        for record in pq.read_table(shard, columns=["id", "audio", "cleaned_text"]).to_pylist():
            total += 1
            same += normalized.get(str(record["id"])) == (sha256(record["audio"]), record["cleaned_text"])
    check(total == len(normalized) == same,
          f"TOZALANGAN_TRAIN_FINAL equals STT_NORMALIZED_FINAL audio and cleaned_text ({same}/{total})")
```

check_hf_sources() · scripts/validate_final_datasets.py · lines 160–170

```python
def check_hf_sources(metadata):
    source_of = {row["audio_path"]: row["source"] for row in read_csv(COMBINED / "provenance/metadata.csv")[1]}
    rows = {row["source_location"]: row for row in metadata}
    source_rows = [item for item in read_csv(COMBINED / "metadata.csv")[1] if source_of[item["audio_path"]] ==
        "hf"]
    mismatch = 0
    for item in source_rows:
        row = rows[f"karakalpak-combined/{item['audio_path']}"]
        mismatch += (FINAL / "hf" / row["audio_path"]).read_bytes() != (COMBINED / item["audio_path"]).read_byt
            es()
        mismatch += row["text_original"] != item["text"] or row["split"] != item["split"]
    check(len(rows) == len(source_rows), f"hf: all {len(source_rows)} HF recordings present, nothing else")
    check(mismatch == 0, "hf: audio bytes, original text and train/test split unchanged")
```

check_leakage() · scripts/validate_final_datasets.py · lines 173–179

```python
def check_leakage(final):
    rows = [row for metadata in final.values() for row in metadata]
    for key in ("pcm_sha256", "text"):
        splits = defaultdict(set)
        for row in rows:
            splits[row[key]].add(row["split"])
        check(all(len(v) == 1 for v in splits.values()), f"no {key} value crosses splits, within or across
            datasets")
```

check_loader() · scripts/validate_final_datasets.py · lines 182–196

```python
def check_loader():
    sys.path.insert(0, str(ROOT / "src"))
    import soundfile as sf
    import config
    import dataset

    loaded = expected = 0
    for name in config.TRAIN_CSVS + config.VALIDATION_CSVS + list(config.TEST_CSVS.values()):
        data = dataset.load_split([name])
        for index in random.Random(0).sample(range(len(data)), 20):
            row = data[index]
            audio, rate = sf.read(row["audio_path"])
            expected += 1
            loaded += rate == 16000 and len(audio) > 0 and bool(row["text"])
    check(loaded == expected, f"src/dataset.py load_split reads {expected} random samples from every training
        CSV")
```

Step 3 · Copy everything to the server

Mac → server

Four `rsync` commands, run in a terminal on the Mac. There is no Python code in this step.

```
cd ~/Documents/IT/repos/"AI & ML"
SERVER="intern@192.168.1.88:/home/intern/AI\ \&\ ML"
rsync -avc --exclude=__pycache__ --exclude=.DS_Store src/ "$SERVER/src/"
rsync -avc scripts/build_final_datasets.py scripts/validate_final_datasets.py
scripts/prepare_benchmarks.py "$SERVER/scripts/"
rsync -avc --delete-after data/final/ "$SERVER/data/final/"
rsync -avc docs/ "$SERVER/docs/"
```

Option	Meaning
`-a`	keep folders, permissions and timestamps
`-v`	list every file it copies
`-c`	compare files by checksum, not size and date: some edited CSVs have exactly the same size as before
`--delete-after`	after copying, delete server files that no longer exist on the Mac (e.g. the old `mendeley/`)
`AI\ \&\ ML`	the Mac's rsync passes the path through the server's shell, so the space and `&` are escaped

Step 4 · Build the character vocabulary

Server · /home/intern/AI & ML

Command	`cd "/home/intern/AI & ML" && source .venv/bin/activate` then `python src/vocab.py`
Time	a few seconds
Reads	`data/final/stt/train.csv`, `data/final/hf/train.csv` (and the validation/test CSVs for a check)
Writes	`outputs/wav2vec2-lora-kaa/vocab.json`

What it does

The model writes text one character at a time, so it needs a fixed list of characters with an id each: the 32 letters of the train transcripts, then `|` for the space between words, `[UNK]` for an unknown character and `[PAD]` for padding, which CTC also uses as its **blank** ("no new character in this frame").

Call chain

```
python src/vocab.py
└─ vocab.main()
   ├─ letters_in(config.TRAIN_CSVS)                the 32 letters
   │  └─ dataset.load_split() → read_split_csv()   pandas → datasets.Dataset
   ├─ build_vocab(letters)                         + | [UNK] [PAD] = 35 tokens
   ├─ save_vocab(vocab)                            outputs/wav2vec2-lora-kaa/vocab.json
   └─ letters_in(VALIDATION_CSVS + TEST_CSVS)      every letter covered?
```

Output

```
Letters: a b c d e f g h i j k l m n o p q r s t u v w x y z á ó ú ı ń ǵ
Vocabulary size: 35
Validation/test letters missing from the vocabulary: none
Saved to: /home/intern/AI & ML/outputs/wav2vec2-lora-kaa/vocab.json
```

Functions

Function	File · lines	What it does
`main()`	vocab.py 22–33	Builds and saves the vocabulary and checks that validation and test use no other characters.
`letters_in(csv_names)`	vocab.py 7–9	Every character in those transcripts except space, sorted.
`build_vocab(letters)`	vocab.py 12–14	Ids 0–31 for the letters, then `|` = 32, `[UNK]` = 33, `[PAD]` = 34.
`save_vocab(vocab)`	vocab.py 17–19	Writes `vocab.json`.
`load_split(csv_names, subset_size=None)`	dataset.py 19–23	Joins split CSVs into a Hugging Face `Dataset` with `audio_path`, `text`, `duration`; optionally a random subset.
`read_split_csv(csv_path)`	dataset.py 7–10	Reads one CSV and makes `audio_path` a full path.
`random_subset(frame, size)`	dataset.py 13–16	Clips up to `SUBSET_MAX_SECONDS`, sampled with `SEED`.
`config.py`	config.py —	Every setting; see Appendix A.

Code

vocab.letters_in() · src/vocab.py · lines 7–9

```
7  def letters_in(csv_names):
8      text = "".join(load_split(csv_names)["text"])
9      return sorted(set(text) - {" "})
```

vocab.build_vocab() · src/vocab.py · lines 12–14

```
12 def build_vocab(letters):
13     tokens = letters + [config.WORD_SEPARATOR, config.UNKNOWN_TOKEN, config.PAD_TOKEN]
14     return {token: index for index, token in enumerate(tokens)}
```

vocab.save_vocab() · src/vocab.py · lines 17–19

```
17 def save_vocab(vocab):
18     config.VOCAB_PATH.parent.mkdir(parents=True, exist_ok=True)
19     config.VOCAB_PATH.write_text(json.dumps(vocab, ensure_ascii=False, indent=2), encoding="utf-8")
```

vocab.main() · src/vocab.py · lines 22–33

```
22 def main():
23     letters = letters_in(config.TRAIN_CSVS)
24     vocab = build_vocab(letters)
25     save_vocab(vocab)
26
27     evaluation_letters = letters_in(config.VALIDATION_CSVS + list(config.TEST_CSVS.values()))
28     missing = [letter for letter in evaluation_letters if letter not in vocab]
29
30     print("Letters:", " ".join(letters))
31     print("Vocabulary size:", len(vocab))
32     print("Validation/test letters missing from the vocabulary:", " ".join(missing) or "none")
33     print("Saved to:", config.VOCAB_PATH)
```

dataset.read_split_csv() · src/dataset.py · lines 7–10

```
7  def read_split_csv(csv_path):
8      frame = pd.read_csv(csv_path, dtype={"text": str}, keep_default_na=False)
9      frame["audio_path"] = [str(csv_path.parent / path) for path in frame["audio_path"]]
10     return frame
```

dataset.random_subset() · src/dataset.py · lines 13–16

```
13 def random_subset(frame, size):
14     if config.SUBSET_MAX_SECONDS:
15         frame = frame[frame["duration"] <= config.SUBSET_MAX_SECONDS]
16     return frame.sample(n=min(size, len(frame)), random_state=config.SEED)
```

dataset.load_split() · src/dataset.py · lines 19–23

```python
def load_split(csv_names, subset_size=None):
    frame = pd.concat([read_split_csv(config.DATA_DIR / name) for name in csv_names], ignore_index=True)
    if subset_size:
        frame = random_subset(frame, subset_size)
    return Dataset.from_pandas(frame, preserve_index=False)
```

Step 5 · Train the model

Server · /home/intern/AI & ML

Command	`nohup python src/train.py > outputs/raw_logs/train_$(date +%F).log 2>&1 &` (keeps running if the SSH session closes; the raw log never overwrites the cleaned train.log) · progress: `tail -c 3000 outputs/raw_logs/train_$(date +%F).log | tr '\r' '\n' | grep "it\]" | tail -1`
Time	2 h 31 min in total for 2,212 steps (553 per epoch × 4) on the NVIDIA GB10, including 4 validation passes (≈ 2 min each)
Reads	`vocab.json` · train and validation CSVs and WAVs · `facebook/wav2vec2-xls-r-300m` (downloaded the first time)
Writes	`checkpoints/checkpoint-N` (best + latest) · `history.csv` · `final_model/` · `results/test_results.json` · `results/predictions_{stt,hf}.csv`

What it does

Fine-tunes the multilingual XLS-R 300M speech model to write Karakalpak. A new 35-token output layer is added, and instead of changing all 300M weights, **LoRA** adds small trainable matrices to every transformer layer: 14.2M trainable parameters (4.3%). Each step learns from 32 clips. After every epoch it measures WER and CER on the 1,950 validation clips and saves a checkpoint; at the end it keeps the best epoch, merges LoRA into the weights, saves `final_model/` and scores the test sets (step 5b). If the run stops, the same command resumes from the last checkpoint.

Call chain

```
python src/train.py
└─ train.main()
   ├─ processor.build_processor()
   │  ├─ build_tokenizer()                                  vocab.json, special tokens from config
   │  └─ build_feature_extractor()                          16 kHz, normalise, attention mask
   ├─ model.build_model(processor)
   │  ├─ load_base_model()                                  XLS-R 300M + CTC head (35)
   │  ├─ freeze_feature_encoder()
   │  └─ add_lora()                                         rank 32 on LORA_TARGET_MODULES
   ├─ dataset.load_split(TRAIN_CSVS)                        17,685 clips, 77.45 h
   ├─ dataset.load_split(VALIDATION_CSVS)                   1,950 clips
   ├─ TrainingLoop(args=training_arguments(), ...)
   │  ├─ data_collator = collator.DataCollatorCTC(processor)
   │  ├─ compute_metrics = metrics.make_compute_metrics(processor)
   │  └─ preprocess_logits_for_metrics = metrics.argmax_logits
   ├─ train.train(resume_from_checkpoint=latest_checkpoint())   2,212 steps
   │  ├─ every batch: audio_batch() + label_batch() → forward → CTC loss → backward
   │  ├─ every epoch: evaluate → argmax_logits → decode() → wer_and_cer()
   │  └─ every epoch: TrainingLoop._save → checkpoints/checkpoint-N
   ├─ save_history() → validation_history()                 history.csv
   ├─ save_final_model()                                    LoRA merged → final_model/
   └─ evaluate_model.evaluate_tests(final_model)            → step 5b
```

Result: validation after each epoch (greedy decoding, 1,950 STT clips)

Epoch	Step	WER	CER	Loss
1	553	85.88%	25.08%	0.948
2	1,106	55.89%	13.07%	0.515
3	1,659	42.77%	9.65%	0.411
4	2,212	38.40%	8.72%	0.381

Inside train.py

What main() does, in order, and what happens inside train.train().

main()

1. `build_processor()`
   processor.py · tokenizer from vocab.json + feature extractor
2. `build_model(processor)`
   model.py · XLS-R 300M + new CTC head (35) + LoRA r=32
3. `load_split(TRAIN_CSVS / VALIDATION_CSVS)`
   dataset.py · 17,685 train · 1,950 validation clips
4. `TrainingLoop(args=training_arguments(), ...)`
   4 epochs · 8 clips × 4 = 32 per step · LR 5e-4 · bf16
5. `train.train(latest_checkpoint())`
   2,212 steps · the loop on the right · resumes if stopped
6. `save_history(log_history)`
   history.csv · WER/CER per epoch
7. `save_final_model(train, processor)`
   LoRA merged into the weights → final_model/
8. `evaluate_tests(final_model)`
   evaluate_model.py · test WER/CER → test_results.json

train.train()

repeats for 2,212 steps (553 per epoch × 4 epochs)

every batch of 8 clips

- DataLoader (8 workers) picks clips of similar length
- DataCollatorCTC: audio_batch() + label_batch()
- forward pass: XLS-R + LoRA → 35 scores per 20 ms frame
- CTC loss → backward (only LoRA + CTC head change)

every 4 batches (32 clips)

- optimizer step · learning-rate schedule

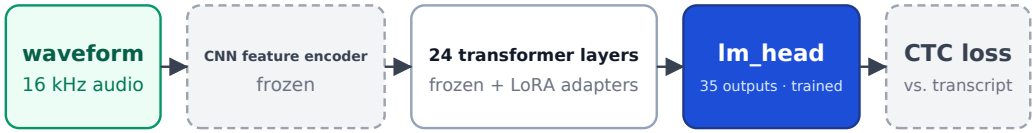
end of each epoch

- evaluate on 1,950 validation clips: argmax_logits → compute_metrics → decode() → wer_and_cer()
- TrainingLoop._save → checkpoints/checkpoint-N

after the last epoch

- reload the checkpoint with the lowest validation WER

The model a batch goes through



LoRA (rank 32) is added to q_proj, k_proj, v_proj, out_proj, intermediate_dense and output_dense in every layer: 14,191,651 trainable of 329,666,246 parameters (4.3%).

Validation after each epoch (1,950 STT clips, greedy decoding)

epoch	step	WER	CER	loss
1	553	85.88%	25.08%	0.948
2	1,106	55.89%	13.07%	0.515
3	1,659	42.77%	9.65%	0.411
4	2,212	**38.40%**	**8.72%**	0.381

Run time on the server: 2 h 31 min for 2,212 steps; the best epoch (4) became final_model/.

Functions

Function	File · lines	What it does
`main()`	train.py 94–119	Builds processor, model and datasets, trains (resuming if a checkpoint exists), then saves history, the merged model and the test scores.
`TrainingLoop._save(...)`	train.py 18–23	Saves a checkpoint: LoRA weights + CTC head, processor, training arguments; `save_embedding_layers=False` because wav2vec2 has no embedding layers.
`training_arguments()`	train.py 26–52	Epochs, batch, learning rate, bf16, grouping by length, evaluate and save each epoch, keep the best by WER.
`latest_checkpoint()`	train.py 55–58	The newest checkpoint, if any, to resume from.
`validation_history(log_history)`	train.py 61–72	Validation loss/WER/CER of every epoch.
`save_history(log_history)`	train.py 75–84	Writes `history.csv` and prints the table.
`save_final_model(trainer, processor)`	train.py 87–91	Merges LoRA and saves `final_model/`.
`build_tokenizer()`	processor.py 6–14	Character ids from `vocab.json`; no `<s>`/`</s>`, so the model has exactly 35 outputs.
`build_feature_extractor()`	processor.py 17–24	16 kHz, each clip normalised, attention mask for padding.
`build_processor()`	processor.py 27–28	Tokenizer + feature extractor.
`load_base_model(processor)`	model.py 17–25	XLS-R 300M with a new 35-output CTC head.
`add_lora(model)`	model.py 28–36	LoRA rank 32 on attention and feed-forward layers; `lm_head` trained in full.
`build_model(processor)`	model.py 39–44	Base model, frozen CNN encoder, LoRA; prints parameter counts.
`gpu_supports_bf16()`	model.py 13–14	True on a GPU with bf16.
`DataCollatorCTC`	collator.py 6–22	`audio_batch` reads, normalises and pads audio; `label_batch` encodes text and marks padding with -100.
`argmax_logits(logits, labels)`	metrics.py 7–8	Keeps only the best token per frame during evaluation.
`replace_ignored_labels(ids, pad_id)`	metrics.py 11–12	-100 back to the pad id.
`decode(processor, pred_ids, label_ids)`	metrics.py 15–19	Greedy CTC decoding of predictions; labels back to text.
`wer_and_cer(references, predictions)`	metrics.py 22–23	WER and CER with `jiwer`.
`make_compute_metrics(processor)`	metrics.py 26–31	What the trainer calls after every evaluation.

Code

train.main() · src/train.py · lines 94–119

```python
def main():
    processor = build_processor()
    model = build_model(processor)

    train_data = load_split(config.TRAIN_CSVS, config.TRAIN_SUBSET_SIZE)
    validation_data = load_split(config.VALIDATION_CSVS, config.EVAL_SUBSET_SIZE)
    print(f"Train: {len(train_data)} files ({sum(train_data['duration']) / 3600:.2f} h)   Validation: {len(vali
        dation_data)} files")

    train = TrainingLoop(
        model=model,
        args=training_arguments(),
        train_dataset=train_data,
        eval_dataset=validation_data,
        data_collator=DataCollatorCTC(processor),
        processing_class=processor,
        compute_metrics=make_compute_metrics(processor),
        preprocess_logits_for_metrics=argmax_logits,
    )

    checkpoint = latest_checkpoint()
    print("Resuming from:", checkpoint or "nothing, starting from scratch")
    train.train(resume_from_checkpoint=checkpoint)

    save_history(train.state.log_history)
    save_final_model(train, processor)
    evaluate_tests(config.FINAL_MODEL_DIR)
```

train.TrainingLoop · src/train.py · lines 17–23

```python
class TrainingLoop(Trainer):
    def _save(self, output_dir=None, state_dict=None):
        output_dir = output_dir or self.args.output_dir
        os.makedirs(output_dir, exist_ok=True)
        self.model.save_pretrained(output_dir, state_dict=state_dict, save_embedding_layers=False)
        self.processing_class.save_pretrained(output_dir)
        torch.save(self.args, os.path.join(output_dir, "training_args.bin"))
```

train.training_arguments() · src/train.py · lines 26–52

```python
def training_arguments():
    return TrainingArguments(
        output_dir=str(config.CHECKPOINT_DIR),
        num_train_epochs=config.EPOCHS,
        per_device_train_batch_size=config.BATCH_SIZE,
        per_device_eval_batch_size=config.BATCH_SIZE,
        gradient_accumulation_steps=config.GRADIENT_ACCUMULATION,
        learning_rate=config.LEARNING_RATE,
        warmup_steps=config.WARMUP_RATIO,
        bf16=gpu_supports_bf16(),
        gradient_checkpointing=config.GRADIENT_CHECKPOINTING,
        gradient_checkpointing_kwargs={"use_reentrant": False},
        train_sampling_strategy="group_by_length",
        length_column_name="duration",
        eval_strategy="epoch",
        save_strategy="epoch",
        save_total_limit=2,
        load_best_model_at_end=True,
        metric_for_best_model="wer",
        greater_is_better=False,
        logging_steps=50,
        dataloader_num_workers=config.NUM_WORKERS,
        remove_unused_columns=False,
        label_names=["labels"],
        report_to="none",
        seed=config.SEED,
    )
```

train.latest_checkpoint() · src/train.py · lines 55–58

```python
def latest_checkpoint():
    if not config.CHECKPOINT_DIR.exists():
        return None
    return get_last_checkpoint(str(config.CHECKPOINT_DIR))
```

train.validation_history() · src/train.py · lines 61–72

```python
def validation_history(log_history):
    history = []
    for log in log_history:
        if "eval_wer" in log:
            history.append({
                "epoch": round(log["epoch"]),
                "step": log["step"],
                "loss": round(log["eval_loss"], 4),
                "wer": round(log["eval_wer"], 4),
                "cer": round(log["eval_cer"], 4),
            })
    return history
```

train.save_history() · src/train.py · lines 75–84

```python
def save_history(log_history):
    history = validation_history(log_history)
    with open(config.OUTPUT_DIR / "history.csv", "w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["epoch", "step", "loss", "wer", "cer"])
        writer.writeheader()
        writer.writerows(history)

    print("\nValidation WER/CER per epoch:")
    for row in history:
        print(f"  epoch {row['epoch']:>2}: WER {row['wer']:.2%}  CER {row['cer']:.2%}  loss {row['loss']}")
```

train.save_final_model() · src/train.py · lines 87–91

```python
def save_final_model(trainer, processor):
    model = trainer.model.merge_and_unload()
    model.save_pretrained(config.FINAL_MODEL_DIR)
    processor.save_pretrained(config.FINAL_MODEL_DIR)
    print("Best model saved to:", config.FINAL_MODEL_DIR)
```

processor.build_tokenizer() · src/processor.py · lines 6–14

```python
def build_tokenizer():
    return Wav2Vec2CTCTokenizer(
        str(config.VOCAB_PATH),
        unk_token=config.UNKNOWN_TOKEN,
        pad_token=config.PAD_TOKEN,
        word_delimiter_token=config.WORD_SEPARATOR,
        bos_token=None,
        eos_token=None,
    )
```

processor.build_feature_extractor() · src/processor.py · lines 17–24

```python
def build_feature_extractor():
    return Wav2Vec2FeatureExtractor(
        feature_size=1,
        sampling_rate=config.SAMPLING_RATE,
        padding_value=0.0,
        do_normalize=True,
        return_attention_mask=True,
    )
```

processor.build_processor() · src/processor.py · lines 27–28

```python
def build_processor():
    return Wav2Vec2Processor(feature_extractor=build_feature_extractor(), tokenizer=build_tokenizer())
```

model.py settings · src/model.py · lines 9–10

```python
LORA_TARGET_MODULES = ["q_proj", "k_proj", "v_proj", "out_proj", "intermediate_dense", "output_dense"]
TRAINED_IN_FULL = ["lm_head"]
```

model.gpu_supports_bf16() · src/model.py · lines 13–14

```python
def gpu_supports_bf16():
    return torch.cuda.is_available() and torch.cuda.is_bf16_supported()
```

model.load_base_model() · src/model.py · lines 17–25

```python
def load_base_model(processor):
    return Wav2Vec2ForCTC.from_pretrained(
        config.BASE_MODEL,
        vocab_size=len(processor.tokenizer),
        pad_token_id=processor.tokenizer.pad_token_id,
        ctc_loss_reduction="mean",
        ctc_zero_infinity=True,
        layerdrop=0.0,
    )
```

model.add_lora() · src/model.py · lines 28–36

```python
def add_lora(model):
    lora_config = LoraConfig(
        r=config.LORA_RANK,
        lora_alpha=config.LORA_ALPHA,
        lora_dropout=config.LORA_DROPOUT,
        target_modules=LORA_TARGET_MODULES,
        modules_to_save=TRAINED_IN_FULL,
    )
    return get_peft_model(model, lora_config)
```

model.build_model() · src/model.py · lines 39–44

```python
def build_model(processor):
    model = load_base_model(processor)
    model.freeze_feature_encoder()
    model = add_lora(model)
    model.print_trainable_parameters()
    return model
```

collator.DataCollatorCTC · src/collator.py · lines 6–22

```python
class DataCollatorCTC:
    def __init__(self, processor):
        self.processor = processor

    def __call__(self, rows):
        batch = self.audio_batch([row["audio_path"] for row in rows])
        batch["labels"] = self.label_batch([row["text"] for row in rows])
        return batch

    def audio_batch(self, paths):
        audio = [sf.read(path, dtype="float32")[0] for path in paths]
        return self.processor.feature_extractor(audio, sampling_rate=config.SAMPLING_RATE, padding=True, return
            _tensors="pt")

    def label_batch(self, texts):
        labels = self.processor.tokenizer(texts, padding=True, return_tensors="pt")
        is_padding = labels["attention_mask"].ne(1)
        return labels["input_ids"].masked_fill(is_padding, config.IGNORED_LABEL_ID)
```

metrics.argmax_logits() · src/metrics.py · lines 7–8

```python
def argmax_logits(logits, labels):
    return logits.argmax(dim=-1)
```

metrics.replace_ignored_labels() · src/metrics.py · lines 11–12

```python
def replace_ignored_labels(ids, pad_id):
    return np.where(ids == config.IGNORED_LABEL_ID, pad_id, ids)
```

metrics.decode() · src/metrics.py · lines 15–19

```python
def decode(processor, pred_ids, label_ids):
    pad_id = processor.tokenizer.pad_token_id
    predictions = processor.tokenizer.batch_decode(replace_ignored_labels(pred_ids, pad_id))
    references = processor.tokenizer.batch_decode(replace_ignored_labels(label_ids, pad_id), group_tokens=False
        )
    return predictions, references
```

metrics.wer_and_cer() · src/metrics.py · lines 22–23

```python
def wer_and_cer(references, predictions):
    return {"wer": jiwer.wer(references, predictions), "cer": jiwer.cer(references, predictions)}
```

metrics.make_compute_metrics() · src/metrics.py · lines 26–31

```python
def make_compute_metrics(processor):
    def compute_metrics(pred):
        predictions, references = decode(processor, pred.predictions, pred.label_ids)
        return wer_and_cer(references, predictions)

    return compute_metrics
```

Step 5b · Score the test sets

Server · automatic at the end of step 5

Command	runs automatically; by hand: `python src/evaluate_model.py [path/to/final_model]`
Time	about 3 minutes
Reads	`final_model/` · `data/final/stt/test.csv` and `data/final/hf/test.csv` with their WAVs
Writes	`results/test_results.json` · `results/predictions_stt.csv` · `results/predictions_hf.csv`

Call chain

```
evaluate_model.evaluate_tests(final_model)
├─ model.load_trained_model(final_model)
├─ prediction_trainer(model, processor)   a Trainer used only to predict
└─ for stt, hf: evaluate_test_set()
   ├─ dataset.load_split([test.csv])
   ├─ trainer.predict(data)               model + WER/CER
   ├─ metrics.decode(...)                 texts for the CSV
   ├─ write_predictions()                 predictions_<set>.csv
   └─ print_examples()
write test_results.json
```

Result

Test set	Clips	WER	CER	Exactly right
STT (news)	1,950	38.43%	8.92%	3
HF (literary)	523	22.32%	3.65%	112 (21%)

Functions

Function	File · lines	What it does
`evaluate_tests(model_dir=None)`	evaluate_model.py 62–71	Every set in `TEST_CSVS`; writes `test_results.json`.
`evaluate_test_set(...)`	evaluate_model.py 46–59	One test set: predict, decode, save the CSV, print WER/CER and 3 examples.
`prediction_trainer(model, processor)`	evaluate_model.py 14–30	A `Trainer` used only to predict.
`write_predictions` · `print_examples`	evaluate_model.py —	Output helpers.
`load_trained_model(model_dir=None)`	model.py 47–49	Loads `final_model/` and its processor; also used by steps 7 and 8.

Code

evaluate_model.evaluate_tests() · src/evaluate_model.py · lines 62–71

```python
def evaluate_tests(model_dir=None):
    model, processor = load_trained_model(model_dir)
    trainer = prediction_trainer(model, processor)
    config.RESULTS_DIR.mkdir(parents=True, exist_ok=True)

    results = {name: evaluate_test_set(trainer, processor, name, csv_name) for name, csv_name in config.TEST_CS
        VS.items()}

    (config.RESULTS_DIR / "test_results.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
    print("\nSaved to:", config.RESULTS_DIR)
    return results
```

evaluate_model.evaluate_test_set() · src/evaluate_model.py · lines 46–59

```python
def evaluate_test_set(trainer, processor, name, csv_name):
    data = load_split([csv_name], config.EVAL_SUBSET_SIZE)
    output = trainer.predict(data, metric_key_prefix=name)
    predictions, references = decode(processor, output.predictions, output.label_ids)
    write_predictions(config.RESULTS_DIR / f"predictions_{name}.csv", data["audio_path"], references, predictio
        ns)

    result = {
        "files": len(data),
        "wer": round(output.metrics[f"{name}_wer"], 4),
        "cer": round(output.metrics[f"{name}_cer"], 4),
    }
    print(f"\n{name} test: WER {result['wer']:.2%}  CER {result['cer']:.2%}  ({len(data)} files)")
    print_examples(references, predictions)
    return result
```

evaluate_model.prediction_trainer() · src/evaluate_model.py · lines 14–30

```python
def prediction_trainer(model, processor):
    args = TrainingArguments(
        output_dir=str(config.RESULTS_DIR),
        per_device_eval_batch_size=config.BATCH_SIZE,
        bf16=gpu_supports_bf16(),
        dataloader_num_workers=config.NUM_WORKERS,
        remove_unused_columns=False,
        label_names=["labels"],
        report_to="none",
    )
    return Trainer(
        model=model,
        args=args,
        data_collator=DataCollatorCTC(processor),
        compute_metrics=make_compute_metrics(processor),
        preprocess_logits_for_metrics=argmax_logits,
    )
```

evaluate_model.write_predictions() · src/evaluate_model.py · lines 33–37

```python
def write_predictions(path, audio_paths, references, predictions):
    with open(path, "w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["audio_path", "reference", "prediction"])
        writer.writerows(zip(audio_paths, references, predictions))
```

evaluate_model.print_examples() · src/evaluate_model.py · lines 40–43

```python
def print_examples(references, predictions, count=3):
    for reference, prediction in list(zip(references, predictions))[:count]:
        print("  reference: ", reference)
        print("  prediction:", prediction)
```

model.load_trained_model() · src/model.py · lines 47–49

```python
def load_trained_model(model_dir=None):
    model_dir = Path(model_dir or config.FINAL_MODEL_DIR)
    return Wav2Vec2ForCTC.from_pretrained(model_dir), Wav2Vec2Processor.from_pretrained(model_dir)
```

Step 6 · Build the KenLM language model

Server · /home/intern/AI & ML

Command	`LD_LIBRARY_PATH=/home/intern/miniforge3/envs/stt/lib python src/lm.py`
Time	a few seconds
Reads	the train, validation and test CSVs (text only)
Writes	`lm/lm_text.txt` · `lm/kaa_4gram.arpa`

What it does

A language model knows which words and word sequences are likely in Karakalpak. `lm.py` takes the 17,685 train sentences minus any that also appear in validation or test (none do), and runs KenLM's `lmplz` to estimate a 4-gram model with modified Kneser–Ney smoothing, its memory capped by `LMPLZ_MEMORY_LIMIT` so it can run next to training. `LD_LIBRARY_PATH` is needed on the server because `lmplz` was compiled against Boost 1.85 from conda, which only the `stt` environment still has.

Call chain

```
python src/lm.py
└─ lm.main()
   ├─ lm_sentences()              train sentences minus validation/test
   │  └─ dataset.load_split() × 2
   ├─ write config.LM_TEXT_PATH   lm/lm_text.txt
   └─ run_lmplz(lm_text.txt, kaa_4gram.arpa)
      └─ subprocess: lmplz -o 4 --discount_fallback -S 1G, into .partial, then renamed
```

Result

n-gram order	1	2	3	4
count	42,014	266,484	408,658	455,587

Checks on the server: 56.3 MB; 4.0% of validation words unknown; perplexity 1,006 on real validation sentences against 8,172 with their words shuffled; with one letter made ambiguous, KenLM fixed it in 50 of 50 sentences (`soqdayaq` → `sondayaq`).

Code

lm.lm_sentences() · src/lm.py · lines 7–12

```
7  def lm_sentences():
8      held_out = set(load_split(config.VALIDATION_CSVS + list(config.TEST_CSVS.values()))["text"])
9      train_text = load_split(config.TRAIN_CSVS)["text"]
10     sentences = [text for text in train_text if text not in held_out]
11     print(f"LM text: {len(sentences)} sentences ({len(train_text) - len(sentences)} removed because they are
       in validation/test)")
12     return sentences
```

lm.run_lmplz() · src/lm.py · lines 15–21

```
15 def run_lmplz(text_path, arpa_path):
16     lmplz = config.KENLM_BIN / "lmplz"
17     command = [str(lmplz), "-o", str(config.LM_ORDER), "--discount_fallback", "-S", config.LMPLZ_MEMORY_LIMIT]
18     unfinished = arpa_path.with_suffix(".partial")
19     with open(text_path, "rb") as source, open(unfinished, "wb") as target:
20         subprocess.run(command, stdin=source, stdout=target, check=True)
21     unfinished.replace(arpa_path)
```

lm.main() · src/lm.py · lines 24–28

```
24 def main():
25     config.LM_DIR.mkdir(parents=True, exist_ok=True)
26     config.LM_TEXT_PATH.write_text("\n".join(lm_sentences()) + "\n", encoding="utf-8")
27     run_lmplz(config.LM_TEXT_PATH, config.LM_PATH)
28     print("Saved to:", config.LM_PATH)
```

Step 7 · Decode and compare: greedy, beam search, KenLM

Server · /home/intern/AI & ML

Command	`python src/decode.py` (or `python src/decode.py path/to/final_model`)
Time	about 4 minutes
Reads	`final_model/` · `lm/kaa_4gram.arpa` · validation and both test CSVs with their WAVs
Writes	`results/decoder_results.json` · `results/decoded_{validation,stt_test,hf_test}.csv`

What it does

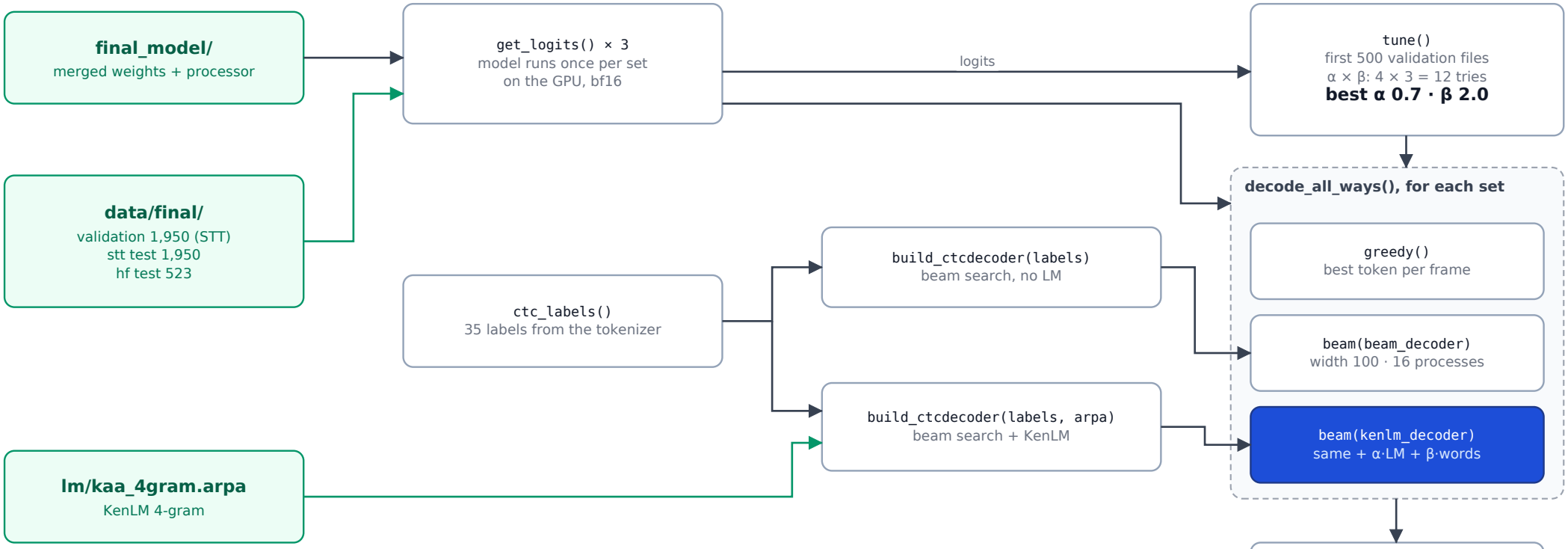
Runs the model once per set and keeps the frame-by-frame scores (logits), then turns the same logits into text three ways. **Greedy** takes the best character in every frame. **Beam search** keeps the 100 best partial transcripts. **Beam search + KenLM** (shallow fusion) scores each one as log P(text | audio) + α · log P_LM(text) + β · words. α and β are tuned on 500 validation files, so the test scores stay fair.

Call chain

```
python src/decode.py
└─ decode.main()
   ├─ model.load_trained_model()
   ├─ ctc_labels(processor)                      35 labels, renamed by PYCTCDECODE_LABELS
   ├─ build_ctcdecoder(labels)                   beam search, no LM (pyctcdecode)
   ├─ build_ctcdecoder(labels, kaa_4gram.arpa)   beam search + KenLM
   ├─ evaluation_sets()                          validation, stt test, hf test
   ├─ get_logits() × 3 → best_device()           the model runs once per set
   ├─ tune(kenlm_decoder, first 500 validation files)
   │  └─ beam() × 12 → wer_and_cer()             α × β grid
   ├─ for each set: decode_all_ways() → greedy(), beam(), beam()
   │  ├─ score()                                 WER / CER
   │  └─ write_decoded()                         decoded_<set>.csv
   └─ print_scores() · decoder_results.json
```

Inside decode.py

The trained model runs once per set; the same logits are then decoded three ways and scored.



Results: WER % / CER %

	greedy	beam	beam + KenLM
validation (1,950)	38.35 / 8.71	37.56 / 8.51	**18.97 / 5.93**
stt test (1,950)	38.37 / 8.90	37.75 / 8.73	**18.61 / 5.93**
hf test (523)	22.32 / 3.65	21.68 / 3.53	**18.30 / 3.08**

KenLM halves WER on STT (38.4% → 18.6%); on HF it drops from 22.3% to 18.3%.

Run time on the server: about 4 minutes.

Shallow fusion: how a beam hypothesis is scored

`score = log P_ctc(text | audio) + α · log P_KenLM(text) + β · words`

α = weight of the language model · β = bonus per word

Result: tuning α and β (WER on 500 validation files)

α \ β	0.0	1.0	2.0
0.3	20.38%	20.96%	21.88%
0.5	19.63%	19.73%	20.05%
0.7	20.17%	19.78%	**19.61%**
1.0	24.47%	22.54%	21.51%

Result: WER / CER by decoder

Set	Greedy	Beam search	Beam search + KenLM
validation (1,950)	38.35% / 8.71%	37.56% / 8.51%	**18.97% / 5.93%**
STT test (1,950)	38.37% / 8.90%	37.75% / 8.73%	**18.61% / 5.93%**
HF test (523)	22.32% / 3.65%	21.68% / 3.53%	**18.30% / 3.08%**

Functions

All in `src/decode.py`.

Function	Lines	What it does
`main(model_dir=None)`	102–127	Loads the model, builds both decoders, computes logits once per set, tunes α/β, decodes and scores every set, writes the results.
`best_device()`	19–24	`cuda`, else `mps`, else `cpu`.
`get_logits(model, processor, data)`	27–39	Runs the model over a set and keeps each file's logits without padding frames.
`ctc_labels(processor)`	42–46	The 35 labels in model-output order for pyctcdecode.
`greedy(processor, logits)`	49–50	Best token per frame.
`beam(decoder, logits)`	53–55	Prefix beam search, width 100, 16 processes.
`decode_all_ways(...)`	58–59	Greedy, beam and beam + KenLM for the same logits.
`score(references, predictions)`	62–63	WER and CER, rounded.
`tune(decoder, logits, references)`	66–78	Every α × β pair; keeps the lowest WER.
`evaluation_sets()`	81–85	Validation and both test sets.
`write_decoded · print_scores`	—	Output helpers, also used by `benchmark.py`.

Code

decode.main() · src/decode.py · lines 102–127

```python
def main(model_dir=None):
    model, processor = load_trained_model(model_dir)
    labels = ctc_labels(processor)
    beam_decoder = build_ctcdecoder(labels)
    kenlm_decoder = build_ctcdecoder(labels, kenlm_model_path=str(config.LM_PATH))

    sets = evaluation_sets()
    logits = {name: get_logits(model, processor, data) for name, data in sets.items()}
    references = {name: list(data["text"]) for name, data in sets.items()}

    tuning_files = min(config.TUNING_FILES, len(sets["validation"]))
    print(f"Tuning KenLM alpha/beta on {tuning_files} validation files")
    alpha, beta = tune(kenlm_decoder, logits["validation"][:tuning_files], references["validation"][:tuning_fil
        es])
    print(f"Best: alpha {alpha}  beta {beta}")

    config.RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    results = {}
    for name, data in sets.items():
        predictions = decode_all_ways(processor, logits[name], beam_decoder, kenlm_decoder)
        results[name] = {decoder: score(references[name], texts) for decoder, texts in predictions.items()}
        write_decoded(config.RESULTS_DIR / f"decoded_{name}.csv", data["audio_path"], references[name], predict
            ions)

    summary = {"beam_width": config.BEAM_WIDTH, "lm": str(config.LM_PATH), "alpha": alpha, "beta": beta, "resul
        ts": results}
    (config.RESULTS_DIR / "decoder_results.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print_scores(results)
    print("\nSaved to:", config.RESULTS_DIR)
```

decode.best_device() · src/decode.py · lines 19–24

```python
def best_device():
    if torch.cuda.is_available():
        return "cuda"
    if torch.backends.mps.is_available():
        return "mps"
    return "cpu"
```

decode.get_logits() · src/decode.py · lines 27–39

```python
def get_logits(model, processor, data):
    device = best_device()
    model.to(device).eval()
    loader = DataLoader(data, batch_size=config.BATCH_SIZE, collate_fn=DataCollatorCTC(processor), num_workers=
        config.NUM_WORKERS)
    all_logits = []

    with torch.no_grad(), torch.autocast(device, dtype=torch.bfloat16, enabled=device == "cuda"):
        for batch in loader:
            logits = model(batch["input_values"].to(device), attention_mask=batch["attention_mask"].to(device))
                .logits
            frames = model._get_feat_extract_output_lengths(batch["attention_mask"].sum(-1))
            all_logits += [row[:length].float().cpu().numpy() for row, length in zip(logits, frames)]

    return all_logits
```

decode.ctc_labels() · src/decode.py · lines 42–46

```python
def ctc_labels(processor):
    labels = [""] * len(processor.tokenizer)
    for token, index in processor.tokenizer.get_vocab().items():
        labels[index] = PYCTCDECODE_LABELS.get(token, token)
    return labels
```

decode.greedy() · src/decode.py · lines 49–50

```python
def greedy(processor, logits):
    return [processor.tokenizer.decode(row.argmax(-1)) for row in logits]
```

decode.beam() · src/decode.py · lines 53–55

```python
def beam(decoder, logits):
    with multiprocessing.get_context("fork").Pool(config.DECODE_PROCESSES) as pool:
        return decoder.decode_batch(pool, logits, beam_width=config.BEAM_WIDTH)
```

decode.decode_all_ways() · src/decode.py · lines 58–59

```
58 def decode_all_ways(processor, logits, beam_decoder, kenlm_decoder):
59     return {"greedy": greedy(processor, logits), "beam": beam(beam_decoder, logits), "kenlm": beam(kenlm_decode
       r, logits)}
```

decode.score() · src/decode.py · lines 62–63

```
62 def score(references, predictions):
63     return {name: round(value, 4) for name, value in wer_and_cer(references, predictions).items()}
```

decode.tune() · src/decode.py · lines 66–78

```
66 def tune(decoder, logits, references):
67     best = None
68     for alpha in config.LM_WEIGHT_CANDIDATES:
69         for beta in config.WORD_BONUS_CANDIDATES:
70             decoder.reset_params(alpha=alpha, beta=beta)
71             wer = wer_and_cer(references, beam(decoder, logits))["wer"]
72             print(f"  alpha {alpha}  beta {beta}:  WER {wer:.2%}")
73             if best is None or wer < best[0]:
74                 best = (wer, alpha, beta)
75
76     _, alpha, beta = best
77     decoder.reset_params(alpha=alpha, beta=beta)
78     return alpha, beta
```

decode.evaluation_sets() · src/decode.py · lines 81–85

```
81 def evaluation_sets():
82     sets = {"validation": load_split(config.VALIDATION_CSVS, config.EVAL_SUBSET_SIZE)}
83     for name, csv_name in config.TEST_CSVS.items():
84         sets[f"{name}_test"] = load_split([csv_name], config.EVAL_SUBSET_SIZE)
85     return sets
```

decode.write_decoded() · src/decode.py · lines 88–92

```
88 def write_decoded(path, audio_paths, references, predictions):
89     with open(path, "w", encoding="utf-8", newline="") as f:
90         writer = csv.writer(f)
91         writer.writerow(["audio_path", "reference", *predictions])
92         writer.writerows(zip(audio_paths, references, *predictions.values()))
```

decode.print_scores() · src/decode.py · lines 95–99

```
95 def print_scores(results):
96     print()
97     for name, scores in results.items():
98         for decoder, s in scores.items():
99             print(f"{name:<12} {decoder:<7} WER {s['wer']:>7.2%}   CER {s['cer']:>7.2%}")
```

Step 8 · Test on data from other projects

Server · /home/intern/AI & ML

Command	`python scripts/prepare_benchmarks.py`, then `python src/benchmark.py data/benchmarks/asr_corpus/test.csv` (and `asr_corpus/validation.csv`, `whisper_test/test.csv`)
Time	seconds to prepare; about 3 minutes per benchmark
Reads	`~/asr_project/data/karakalpak_speech_corpus` (parquet, OGG audio) · `~/whisper_kaa/manifest_tayyor.csv`, its test FLAC files and `07_eval/test_finetune_beam5.csv` · `final_model/` · the LM · `decoder_results.json`
Writes	`data/benchmarks/{asr_corpus,whisper_test}/` (CSVs, metadata, OGG audio) · `results/benchmarks/<name>.json` and `<name>.csv`

What it does

Other projects on the server hold Karakalpak test data. Only sets whose sentences are not in our training text count as new: the asr_corpus test and validation sets and the whisper_kaa test set. `prepare_benchmarks.py` cleans their references like our training text and repairs a flaw in the asr_corpus references, which wrote every hyphen between two words as the word **"alıw"** ("minus"). `benchmark.py` then scores the trained model with all three decoders, using α and β from step 7.

Call chain

```
python scripts/prepare_benchmarks.py
├─ asr_corpus()                          test + validation parquet
│  ├─ save_audio()                       OGG bytes unchanged
│  ├─ restore_hyphens_written_as_minus()   hayal alıw qızlardıń → hayal qızlardıń
│  ├─ benchmark_row() → normalize() → clean_text()
│  │  └─ exclusion_reason()              doubtful alıw, characters outside the alphabet
│  └─ write_benchmark() → write_rows()   <split>.csv + <split>_metadata.csv
└─ whisper_test()                        FLAC paths + the Whisper model's predictions
   └─ benchmark_row() · write_benchmark()

python src/benchmark.py data/benchmarks/asr_corpus/test.csv
└─ benchmark.main()
   ├─ tuned_alpha_and_beta()             α, β from decoder_results.json
   ├─ model.load_trained_model()
   ├─ decode.ctc_labels() · build_ctcdecoder() × 2
   ├─ dataset.read_split_csv(test.csv)
   ├─ decode.get_logits() → decode.decode_all_ways()
   ├─ decode.score()                     WER / CER per decoder
   └─ decode.write_decoded() · decode.print_scores() · <name>.json
```

Result: WER / CER on new sentences

Benchmark	Clips	Greedy	Beam	Beam + KenLM
asr_corpus test	1,047	39.59% / 9.06%	38.71% / 8.88%	**21.32% / 6.25%**
asr_corpus validation	1,048	38.16% / 8.69%	37.34% / 8.51%	**20.27% / 5.88%**
whisper_kaa test	1,018	38.56% / 8.80%	37.65% / 8.60%	**20.64% / 6.04%**

Comparison on the same clips and references	Their model	Ours (beam + KenLM)
Colleague's fine-tuned Whisper-small, whisper_kaa test (1,018)	20.16% / 7.11%	20.64% / 6.04%
Model saved in `~/Benchmark`, HF test (508)	32.76% / 6.89% (KenLM)	18.38% / 2.98%

The asr_corpus references split hyphenated words while our training text joins them; scored with joined words, the asr_corpus test WER would be 19.9% instead of 21.3%.

Functions

Function	File · lines	What it does
`asr_corpus()`	prepare_benchmarks.py 96–107	Test and validation sets from the asr_project parquet files.
`whisper_test()`	prepare_benchmarks.py 110–120	The whisper_kaa test FLAC files with references and Whisper's predictions.
`save_audio(folder, split, record)`	prepare_benchmarks.py 87–93	Writes the OGG bytes unchanged; returns path and duration.
`restore_hyphens_written_as_minus(raw_text, text)`	prepare_benchmarks.py 37–44	Turns `left alıw right` back into `left right` for every hyphenated pair in `raw_text`; flags rows still in doubt.
`normalize(text)` · `without_punctuation(text)`	prepare_benchmarks.py 32–34	Lowercase, foreign letters folded, no punctuation, then `clean_text`.
`exclusion_reason(text, still_doubtful)`	prepare_benchmarks.py 47–53	Why a row is left out, or empty.
`benchmark_row(...)`	prepare_benchmarks.py 56–68	One metadata row: cleaned reference, rules, exclusion, other model's prediction.
`write_benchmark` · `write_rows`	prepare_benchmarks.py —	`<name>.csv` (kept rows) and `<name>_metadata.csv` (all rows).
`main(csv_path, name=None)`	benchmark.py 19–42	Scores the model on one benchmark CSV with all three decoders and writes the results.
`tuned_alpha_and_beta()`	benchmark.py 14–16	α and β from step 7's `decoder_results.json`.

Code

prepare_benchmarks.py settings · scripts/prepare_benchmarks.py · lines 15–25

```python
ROOT = Path(__file__).resolve().parents[1]
BENCHMARKS = ROOT / "data" / "benchmarks"
ASR_CORPUS = Path.home() / "asr_project" / "data" / "karakalpak_speech_corpus" / "parquet_files"
WHISPER_PROJECT = Path.home() / "whisper_kaa"

FOREIGN_LETTERS = str.maketrans({"é": "e", "ü": "ú", "ö": "ó"})
HYPHENATED_WORD_PAIR = re.compile(r"([^\W\d_]+)-(?=([^\W\d_]+))")
MINUS_WORD = "alıw"

CSV_FIELDS = ["audio_path", "text", "duration"]
METADATA_FIELDS = [*CSV_FIELDS, "source_id", "text_original", "text_rules", "excluded", "other_prediction"]
```

asr_corpus() · scripts/prepare_benchmarks.py · lines 96–107

```python
def asr_corpus():
    folder = BENCHMARKS / "asr_corpus"
    for split in ("test", "validation"):
        rows = []
        for shard in sorted(ASR_CORPUS.glob(f"{split}-*.parquet")):
            for record in pq.read_table(shard, columns=["id", "cleaned_audio", "raw_text", "text"]).to_pylist()
                :
                audio_path, duration = save_audio(folder, split, record)
                reference, restored, still_doubtful = restore_hyphens_written_as_minus(record["raw_text"], reco
                    rd["text"])
                extra_rule = f"hyphen-alıw×{restored}" if restored else ""
                rows.append(benchmark_row(audio_path, record["text"], duration, record["id"], reference,
                                          still_doubtful, extra_rule))
        write_benchmark(folder, split, rows)
```

whisper_test() · scripts/prepare_benchmarks.py · lines 110–120

```python
def whisper_test():
    manifest = pd.read_csv(WHISPER_PROJECT / "manifest_tayyor.csv", keep_default_na=False)
    predictions = pd.read_csv(WHISPER_PROJECT / "07_eval" / "test_finetune_beam5.csv", keep_default_na=False)
    whisper_prediction = dict(zip(predictions["idx"], predictions["hypothesis"]))
    rows = []
    for item in manifest[manifest["split"] == "test"].itertuples():
        path = WHISPER_PROJECT / item.fayl.replace("\\", "/")
        if path.exists():
            rows.append(benchmark_row(str(path), item.matn_model, float(item.yangi_s), item.idx,
                                      other_prediction=whisper_prediction.get(item.idx, "")))
    write_benchmark(BENCHMARKS / "whisper_test", "test", rows)
```

save_audio() · scripts/prepare_benchmarks.py · lines 87–93

```python
def save_audio(folder, split, record):
    data = record["cleaned_audio"]["bytes"]
    suffix = Path(record["cleaned_audio"]["path"]).suffix or ".ogg"
    audio_path = f"{split}/audio/{record['id']}{suffix}"
    (folder / audio_path).parent.mkdir(parents=True, exist_ok=True)
    (folder / audio_path).write_bytes(data)
    return audio_path, sf.info(io.BytesIO(data)).duration
```

restore_hyphens_written_as_minus() · scripts/prepare_benchmarks.py · lines 37–44

```python
def restore_hyphens_written_as_minus(raw_text, text):
    restored = 0
    for left, right in HYPHENATED_WORD_PAIR.findall(raw_text.lower()):
        fixed = text.replace(f"{left} {MINUS_WORD} {right}", f"{left} {right}", 1)
        restored += fixed != text
        text = fixed
    still_doubtful = text.count(MINUS_WORD) > raw_text.lower().count(MINUS_WORD)
    return text, restored, still_doubtful
```

normalize() · scripts/prepare_benchmarks.py · lines 32–34

```python
def normalize(text):
    text = unicodedata.normalize("NFC", str(text)).lower().translate(FOREIGN_LETTERS)
    return clean_text(without_punctuation(text))
```

without_punctuation() · scripts/prepare_benchmarks.py · lines 28–29

```python
def without_punctuation(text):
    return " ".join("".join(c if c.isalnum() or c.isspace() else " " for c in text).split())
```

exclusion_reason() · scripts/prepare_benchmarks.py · lines 47–53

```python
def exclusion_reason(text, still_doubtful):
    if still_doubtful:
        return "possible false alıw from a hyphen"
    outside = sorted(set(text) - ALPHABET)
    if outside or not text:
        return f"characters outside the alphabet: {' '.join(outside)}"
    return ""
```

benchmark_row() · scripts/prepare_benchmarks.py · lines 56–68

```python
def benchmark_row(audio_path, text_original, duration, source_id, reference=None, still_doubtful=False,
                  extra_rule="", other_prediction=""):
    text, rules = normalize(text_original if reference is None else reference)
    return {
        "audio_path": audio_path,
        "text": text,
        "duration": duration,
        "source_id": source_id,
        "text_original": text_original,
        "text_rules": ";".join(rule for rule in (extra_rule, rules) if rule),
        "excluded": exclusion_reason(text, still_doubtful),
        "other_prediction": normalize(other_prediction)[0] if other_prediction else "",
    }
```

write_benchmark() · scripts/prepare_benchmarks.py · lines 78–84

```python
def write_benchmark(folder, name, rows):
    kept = [row for row in rows if not row["excluded"]]
    folder.mkdir(parents=True, exist_ok=True)
    write_rows(folder / f"{name}.csv", kept, CSV_FIELDS)
    write_rows(folder / f"{name}_metadata.csv", rows, METADATA_FIELDS)
    hours = sum(row["duration"] for row in kept) / 3600
    print(f"{folder.name}/{name}.csv: {len(kept)} clips ({len(rows) - len(kept)} excluded), {hours:.2f} h")
```

write_rows() · scripts/prepare_benchmarks.py · lines 71–75

```python
def write_rows(path, rows, fields):
    with path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields, extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)
```

benchmark.tuned_alpha_and_beta() · src/benchmark.py · lines 14–16

```python
def tuned_alpha_and_beta():
    tuned = json.loads((config.RESULTS_DIR / "decoder_results.json").read_text(encoding="utf-8"))
    return tuned["alpha"], tuned["beta"]
```

benchmark.main() · src/benchmark.py · lines 19–42

```python
def main(csv_path, name=None):
    csv_path = Path(csv_path).resolve()
    name = name or f"{csv_path.parent.name}_{csv_path.stem}"
    alpha, beta = tuned_alpha_and_beta()

    model, processor = load_trained_model()
    labels = ctc_labels(processor)
    beam_decoder = build_ctcdecoder(labels)
    kenlm_decoder = build_ctcdecoder(labels, kenlm_model_path=str(config.LM_PATH), alpha=alpha, beta=beta)

    data = Dataset.from_pandas(read_split_csv(csv_path), preserve_index=False)
    references = list(data["text"])
    predictions = decode_all_ways(processor, get_logits(model, processor, data), beam_decoder, kenlm_decoder)
    results = {decoder: score(references, texts) for decoder, texts in predictions.items()}

    config.BENCHMARK_RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    write_decoded(config.BENCHMARK_RESULTS_DIR / f"{name}.csv", data["audio_path"], references, predictions)
    hours = round(sum(data["duration"]) / 3600, 2)
    summary = {"benchmark": str(csv_path), "files": len(data), "hours": hours, "alpha": alpha, "beta": beta, "r
        esults": results}
    (config.BENCHMARK_RESULTS_DIR / f"{name}.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")

    print(f"\n{name}: {len(data)} files, {hours} h (alpha {alpha}, beta {beta} from our validation set)")
    print_scores({name: results})
    print("Saved to:", config.BENCHMARK_RESULTS_DIR)
```

Appendix A · config.py: every setting

Every `src/` file imports `config` and reads `config.X`, so a value changed here changes everywhere. `LOCAL=1` before a command makes `RUN_ON_MAC` true and switches to the small local preset (Appendix B).

Setting	Value	Meaning
`DATA_DIR`, `OUTPUT_DIR`	data/final · outputs/wav2vec2-lora-kaa	where the CSVs are read; every output path is inside `OUTPUT_DIR`
`TRAIN_CSVS`, `VALIDATION_CSVS`, `TEST_CSVS`	stt + hf train · stt validation · stt, hf	which split CSVs train, pick the best epoch, and are scored at the end
`SAMPLING_RATE`	16000	all audio is 16 kHz
`WORD_SEPARATOR`, `UNKNOWN_TOKEN`, `PAD_TOKEN`	`|`, `[UNK]`, `[PAD]`	special tokens; `[PAD]` is the CTC blank
`IGNORED_LABEL_ID`	-100	padding value CTC loss skips
`BASE_MODEL`	wav2vec2-xls-r-300m	pretrained multilingual speech encoder
`EPOCHS`	4	passes over the training data
`BATCH_SIZE` × `GRADIENT_ACCUMULATION`	8 × 4	32 clips per optimizer step
`LEARNING_RATE`, `WARMUP_RATIO`	5e-4, 10%	step size, with a gentle start
`GRADIENT_CHECKPOINTING`	False	True saves memory, ~30% slower (used on the Mac)
`LORA_RANK`, `LORA_ALPHA`, `LORA_DROPOUT`	32, 64, 0.05	size and strength of the LoRA adapters
`NUM_WORKERS`, `SEED`	8, 42	processes loading audio in parallel · fixed randomness
`TRAIN_SUBSET_SIZE`, `EVAL_SUBSET_SIZE`, `SUBSET_MAX_SECONDS`	None	optional subset for quick runs
`KENLM_BIN`, `LMPLZ_MEMORY_LIMIT`, `LM_ORDER`, `LM_PATH`	kenlm/build/bin, 1G, 4, lm/kaa_4gram.arpa	language model
`BEAM_WIDTH`, `LM_WEIGHT_CANDIDATES`, `WORD_BONUS_CANDIDATES`, `TUNING_FILES`	100, 4 α, 3 β, 500	decoder search and tuning
`DECODE_PROCESSES`	16	CPU processes for beam search

config.py (whole file) · src/config.py · lines 1–67

```python
import os
from pathlib import Path


ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data" / "final"
TRAIN_CSVS = ["stt/train.csv", "hf/train.csv"]
VALIDATION_CSVS = ["stt/validation.csv"]
TEST_CSVS = {"stt": "stt/test.csv", "hf": "hf/test.csv"}

SAMPLING_RATE = 16000
WORD_SEPARATOR = "|"
UNKNOWN_TOKEN = "[UNK]"
PAD_TOKEN = "[PAD]"
IGNORED_LABEL_ID = -100

BASE_MODEL = "facebook/wav2vec2-xls-r-300m"
OUTPUT_DIR = ROOT / "outputs" / "wav2vec2-lora-kaa"

EPOCHS = 4
BATCH_SIZE = 8
GRADIENT_ACCUMULATION = 4
LEARNING_RATE = 5e-4
WARMUP_RATIO = 0.1
GRADIENT_CHECKPOINTING = False

LORA_RANK = 32
LORA_ALPHA = 64
LORA_DROPOUT = 0.05

NUM_WORKERS = 8
SEED = 42

TRAIN_SUBSET_SIZE = None
EVAL_SUBSET_SIZE = None
SUBSET_MAX_SECONDS = None

RUN_ON_MAC = bool(os.environ.get("LOCAL"))
if RUN_ON_MAC:
    OUTPUT_DIR = ROOT / "outputs" / "wav2vec2-lora-kaa-local"
    TRAIN_SUBSET_SIZE = 3000
    EVAL_SUBSET_SIZE = 300
    SUBSET_MAX_SECONDS = 20
    EPOCHS = 5
    BATCH_SIZE = 4
    GRADIENT_ACCUMULATION = 4
    NUM_WORKERS = 0
    GRADIENT_CHECKPOINTING = True

VOCAB_PATH = OUTPUT_DIR / "vocab.json"
CHECKPOINT_DIR = OUTPUT_DIR / "checkpoints"
FINAL_MODEL_DIR = OUTPUT_DIR / "final_model"
RESULTS_DIR = OUTPUT_DIR / "results"
BENCHMARK_RESULTS_DIR = RESULTS_DIR / "benchmarks"

KENLM_BIN = ROOT / "kenlm" / "build" / "bin"
LMPLZ_MEMORY_LIMIT = "1G"
LM_ORDER = 4
LM_DIR = OUTPUT_DIR / "lm"
LM_TEXT_PATH = LM_DIR / "lm_text.txt"
LM_PATH = LM_DIR / f"kaa_{LM_ORDER}gram.arpa"

BEAM_WIDTH = 100
LM_WEIGHT_CANDIDATES = [0.3, 0.5, 0.7, 1.0]
WORD_BONUS_CANDIDATES = [0.0, 1.0, 2.0]
TUNING_FILES = 500
DECODE_PROCESSES = 16
```

Appendix B · Quick local run on the Mac

The same code trains a small model on the Mac (Apple M5 Pro, 24 GB) to check the pipeline. `LOCAL=1` switches `config.py` to 3,000 random train clips of at most 20 s, 300 clips per evaluation set, 5 epochs, batches of 4 × 4, gradient checkpointing and no loader processes; results go to `outputs/wav2vec2-lora-kaa-local/`.

```
cd ~/Documents/IT/repos/"AI & ML"
export LOCAL=1 HF_HUB_CACHE="$PWD/checkpoints/hf-cache/hub"
mkdir -p outputs/raw_logs
.venv/bin/python src/vocab.py
nohup .venv/bin/python src/train.py > outputs/raw_logs/train_local_$(date +%F).log 2>&1 &
```

Run	Train clips	HF test WER / CER	STT test WER / CER
Mac (local, 5 epochs)	3,000	74.57% / 15.82% (300 clips)	90.29% / 29.56% (300 clips)
Server (4 epochs)	17,685	22.32% / 3.65% (523 clips)	38.43% / 8.92% (1,950 clips)

Appendix C · Results summary

	Value
Training data	17,685 clips, 77.45 h (STT 15,595 + HF 2,090)
Model	XLS-R 300M + LoRA r=32 · 14.2M trainable of 329.7M · 35-token CTC head
Training	4 epochs · 2,212 steps · 2 h 31 min on NVIDIA GB10
Best validation (greedy)	WER 38.40% · CER 8.72% (epoch 4)
HF test	greedy 22.32% / 3.65% · beam 21.68% / 3.53% · **beam + KenLM 18.30% / 3.08%**
STT test	greedy 38.37% / 8.90% · beam 37.75% / 8.73% · **beam + KenLM 18.61% / 5.93%**
New sentences (beam + KenLM)	asr_corpus test 21.32% / 6.25% · validation 20.27% / 5.88% · whisper_kaa test 20.64% / 6.04%
Against other models	Whisper-small 20.16% / 7.11% on its test set · ~/Benchmark model 32.76% / 6.89% on HF test
KenLM	4-gram from the train text · α 0.7 · β 2.0 · beam width 100